

Deep Learning for Bio-image Analysis

Robert Haase

Funded by



Bundesministerium
für Bildung
und Forschung

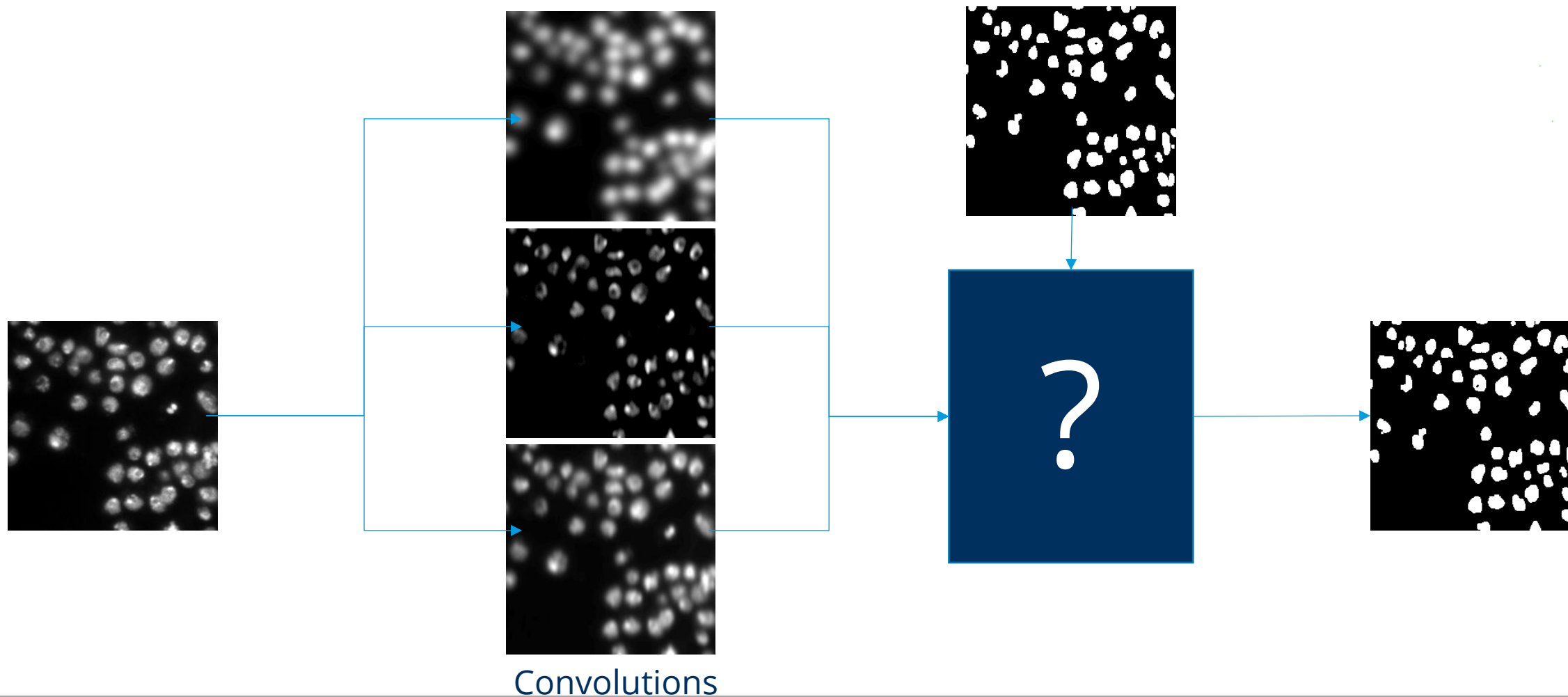
SACHSEN



Diese Maßnahme wird gefördert durch die Bundesregierung
aufgrund eines Beschlusses des Deutschen Bundestages.
Diese Maßnahme wird mitfinanziert durch Steuermittel auf
der Grundlage des von den Abgeordneten des Sächsischen
Landtags beschlossenen Haushaltes.

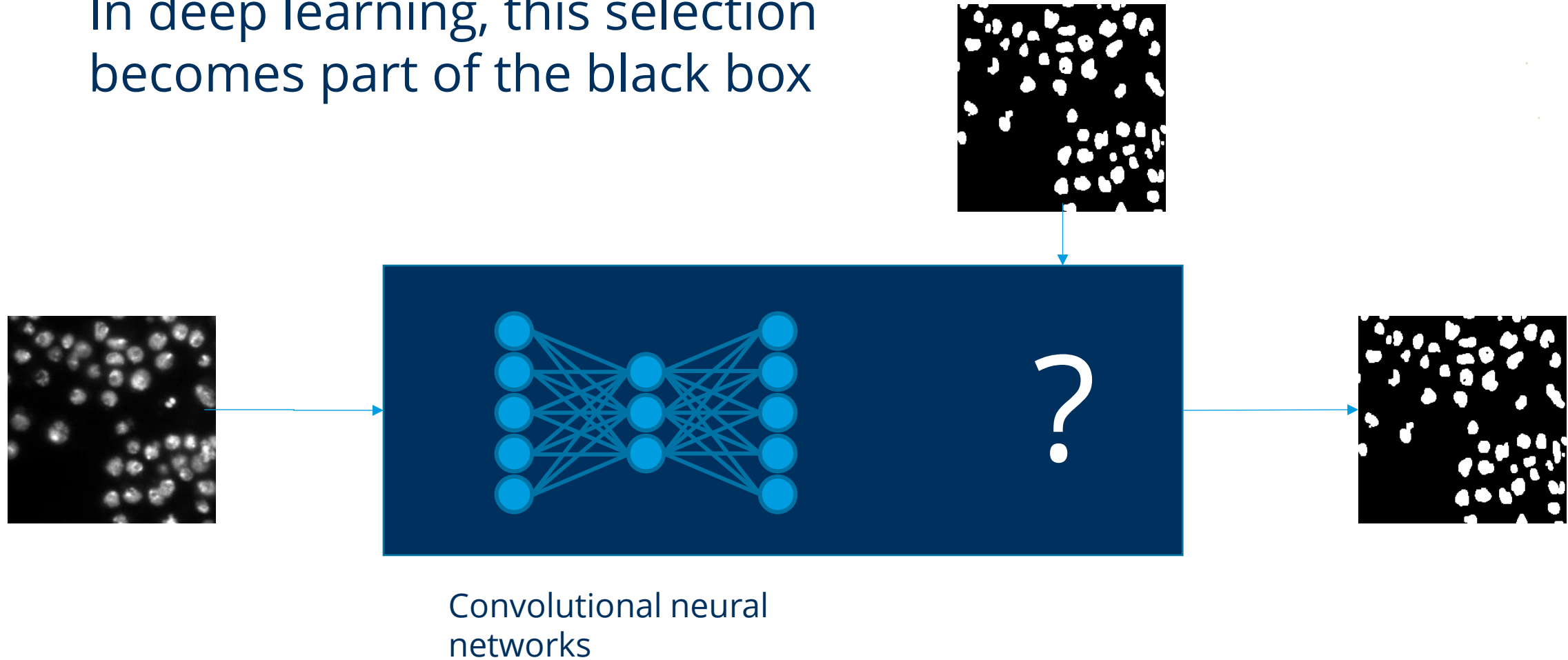
Machine learning for image analysis

In classical machine learning, we typically select features for training our classifier



Outlook: Deep learning for image analysis

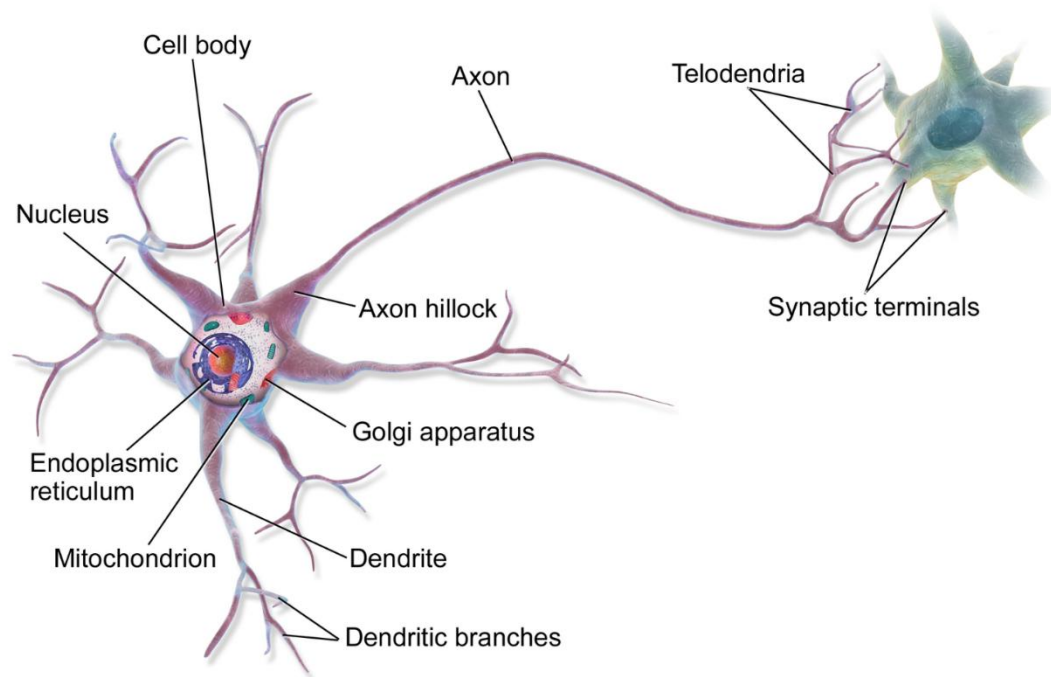
In deep learning, this selection becomes part of the black box



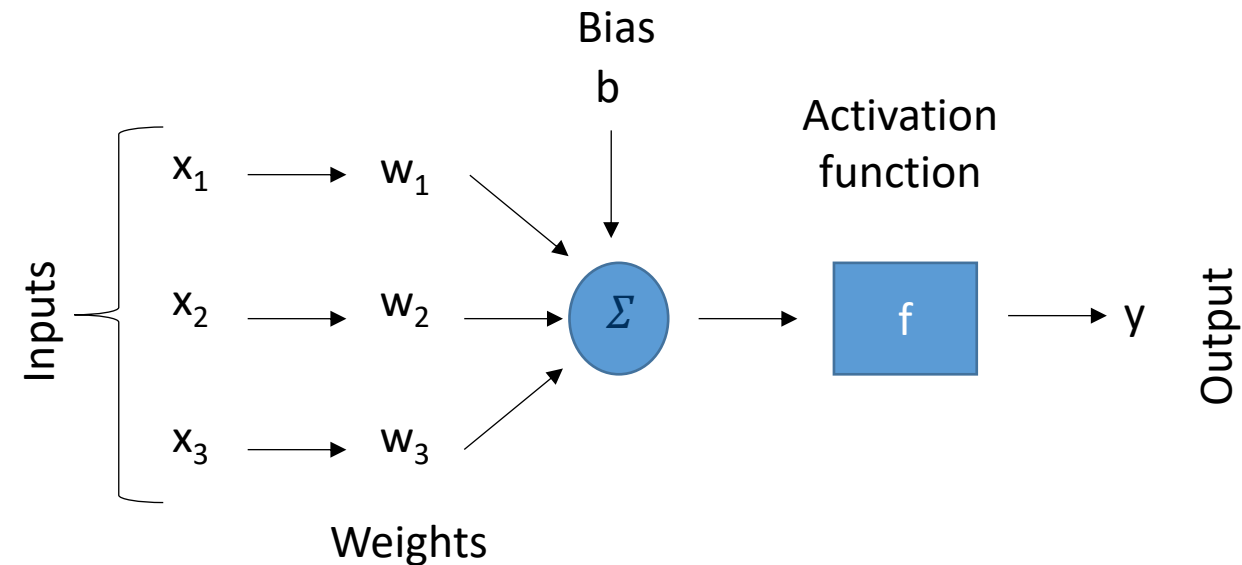
Convolutional neural
networks

Neural networks

- How biologists see neurons



- How computer scientists see neurons
“perceptron”



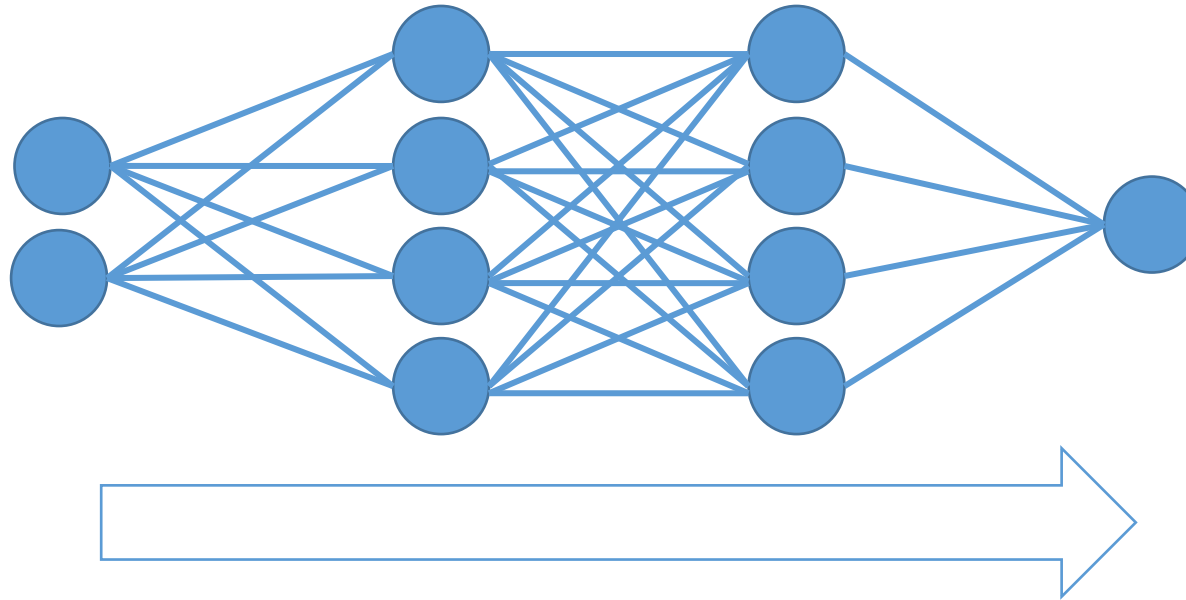
$$y = f(w | 1x_1 + w_2x_2 + w_3x_3 + b)$$

Neural Networks

- Early form: “Multilayer Perceptron”
- fully connected class of feedforward artificial neural network

If there are *many* hidden layers, we speak of a *deep* neural network

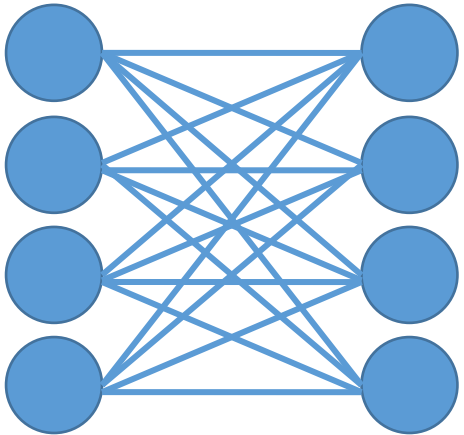
Input layer Hidden layer(s) Output layer



Convolutional neural networks

- Layer types

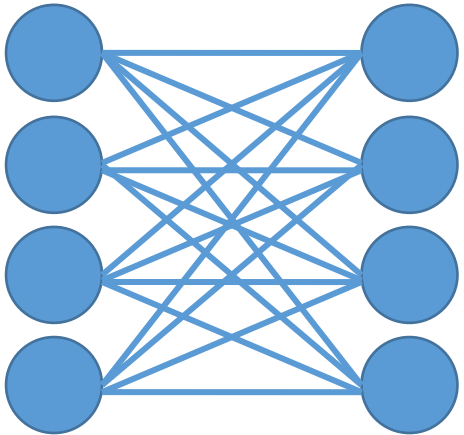
Fully connected layer



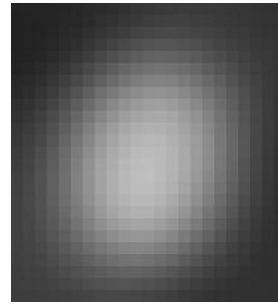
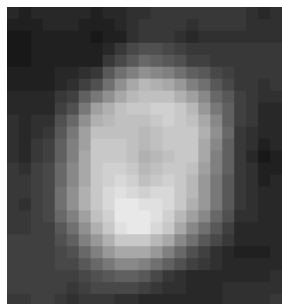
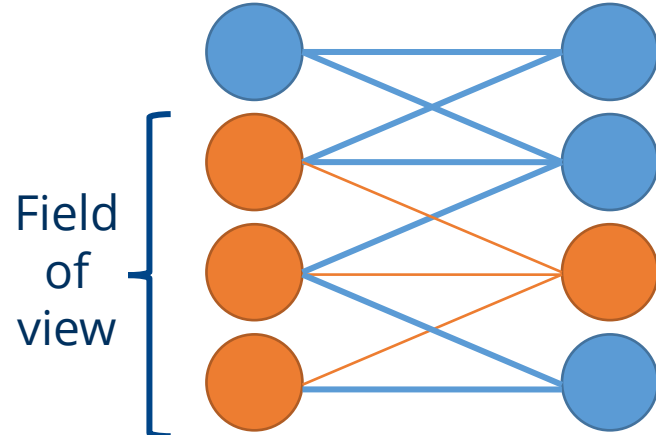
Convolutional neural networks

- Layer types

Fully connected layer



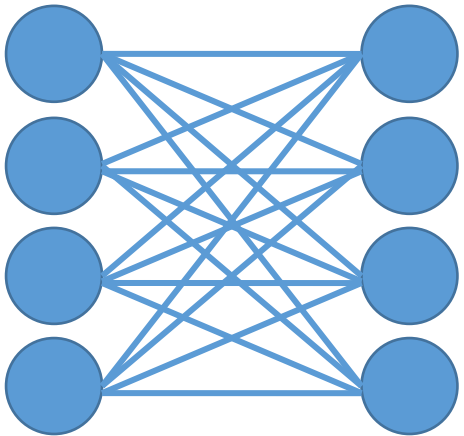
Convolutional layer



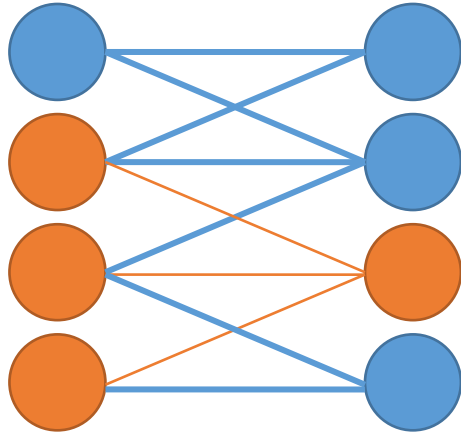
Convolutional neural networks

- Layer types

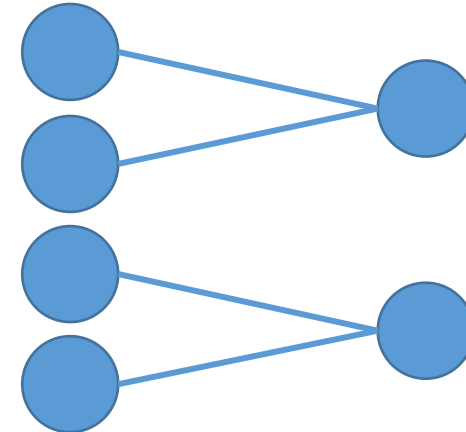
Fully connected layer



Convolutional layer



Pooling layer
("Max pool", "Average pool")



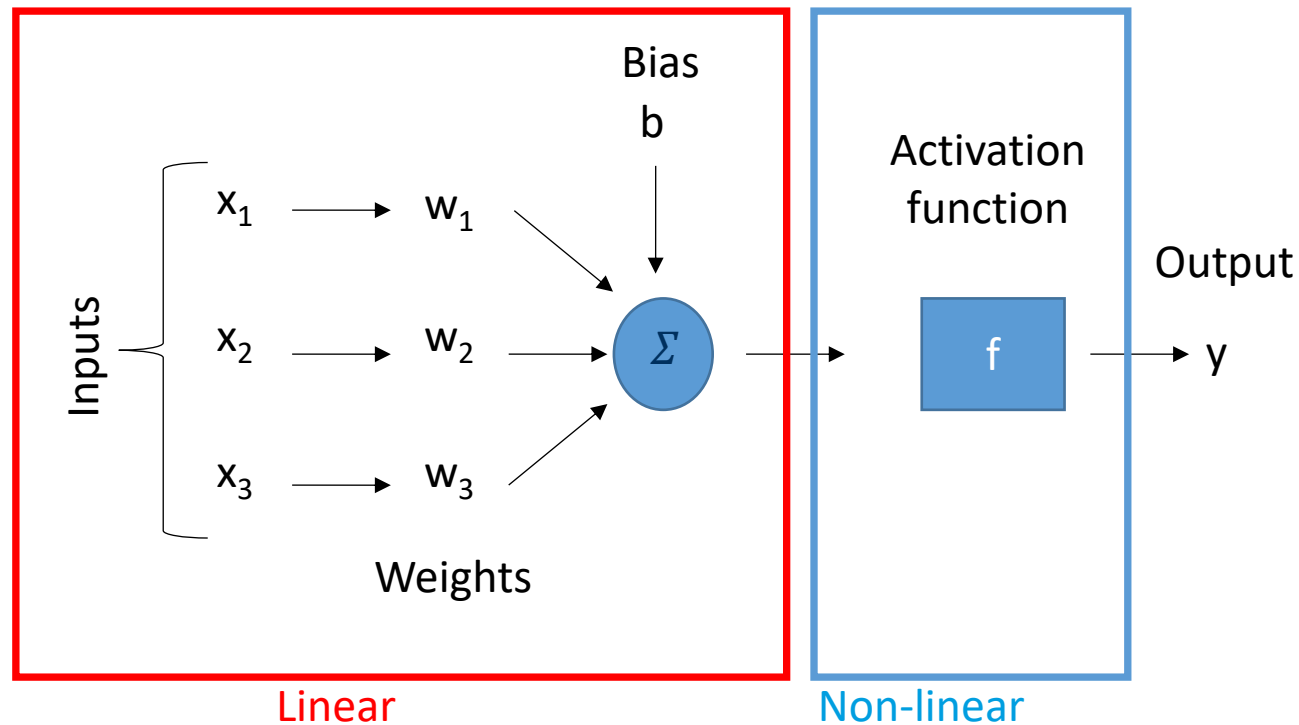
3	15	1	13
9	7	0	10
11	5	5	3
1	8	9	6

Max pooling

15	13
11	9

Activation functions

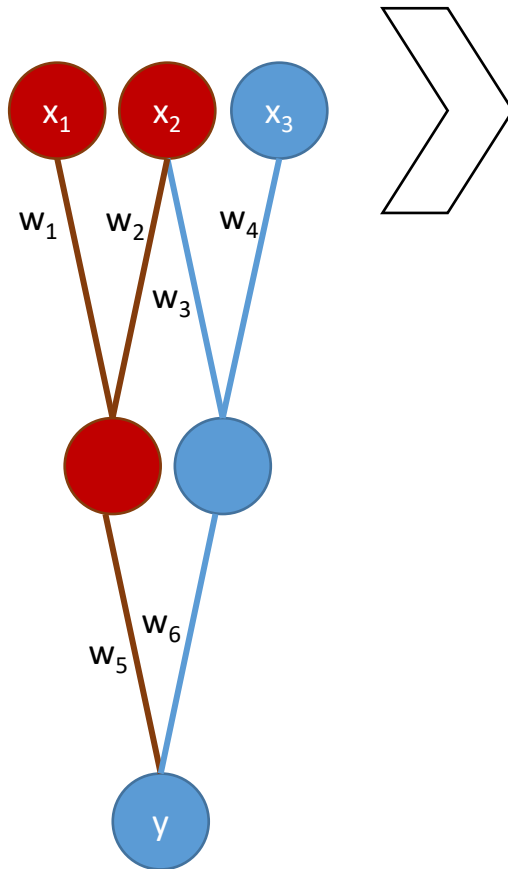
- Introduction of *non-linearity* and *activation functions* enabled what we call *deep-learning* today.



$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

Convolutional neural networks

- Assuming we had no activation functions in the network layers can be reduced by eliminating brackets!



$$y = w_5(w_1x_1 + w_2x_2) + w_6(w_3x_2 + w_4x_3)$$

$$y = w_5w_1x_1 + w_5w_2x_2 + w_6w_3x_2 + w_6w_4x_3$$

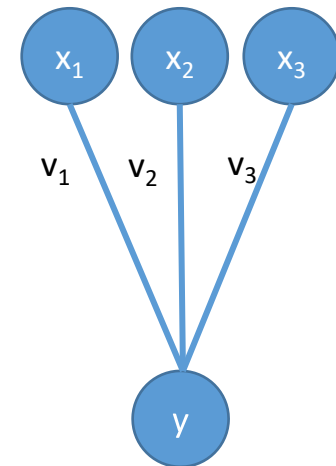
$$y = w_5w_1x_1 +$$

$$v_1 = w_5w_1$$

$$v_2 = w_5w_2 + w_6w_3$$

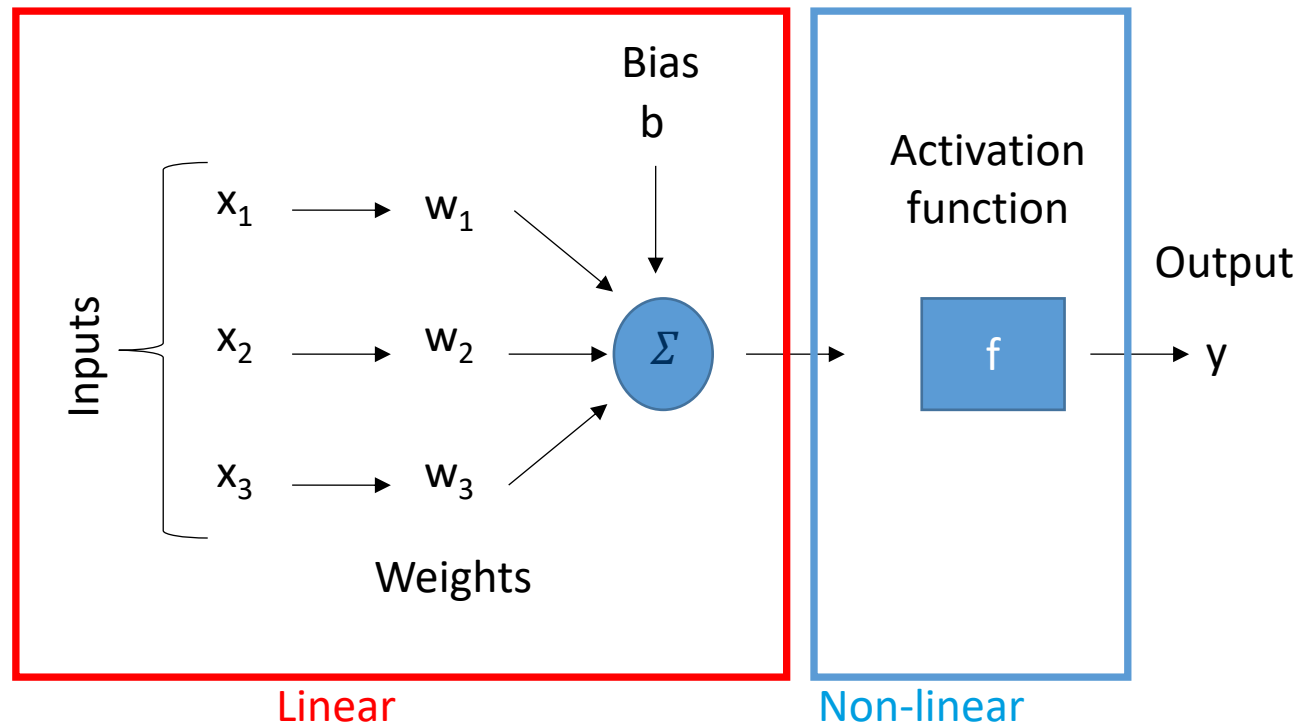
$$v_3 = w_6w_4$$

$$y = v_1x_1 + v_2x_2 + v_3x_3$$



Activation functions

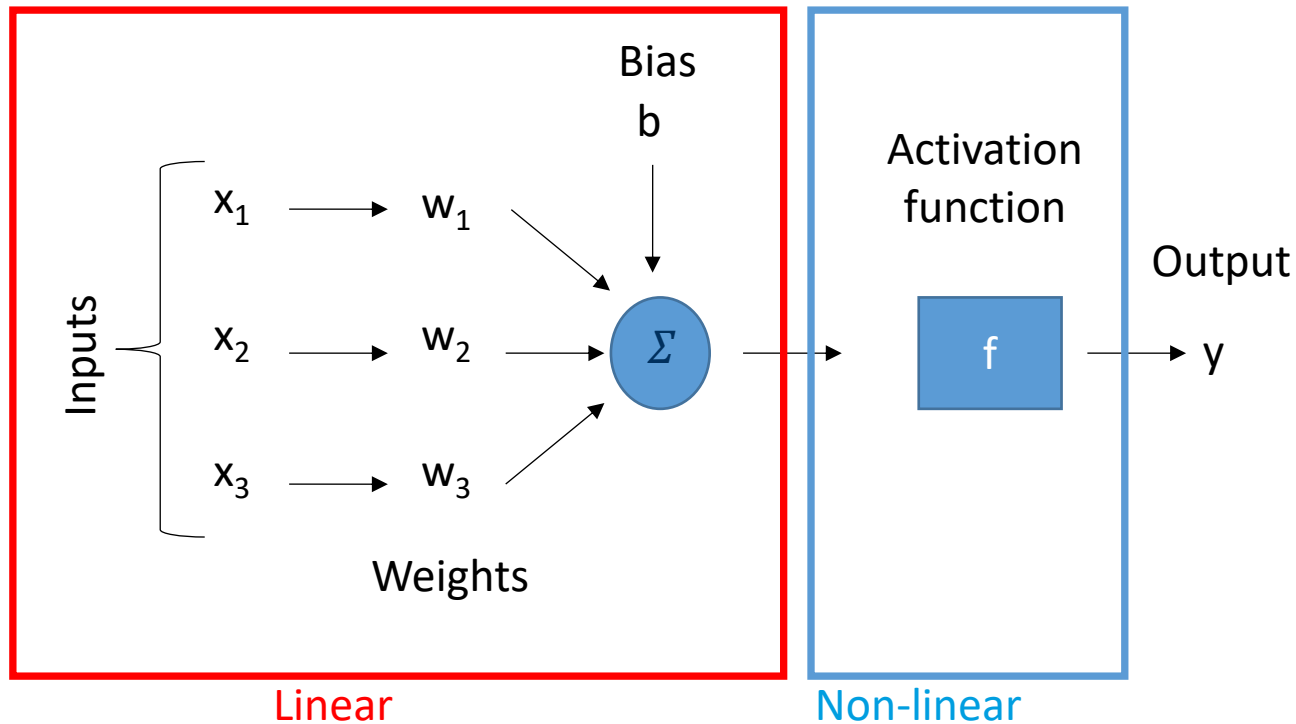
- Introduction of *non-linearity* and *activation functions* enabled what we call *deep-learning* today.



$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

Activation functions

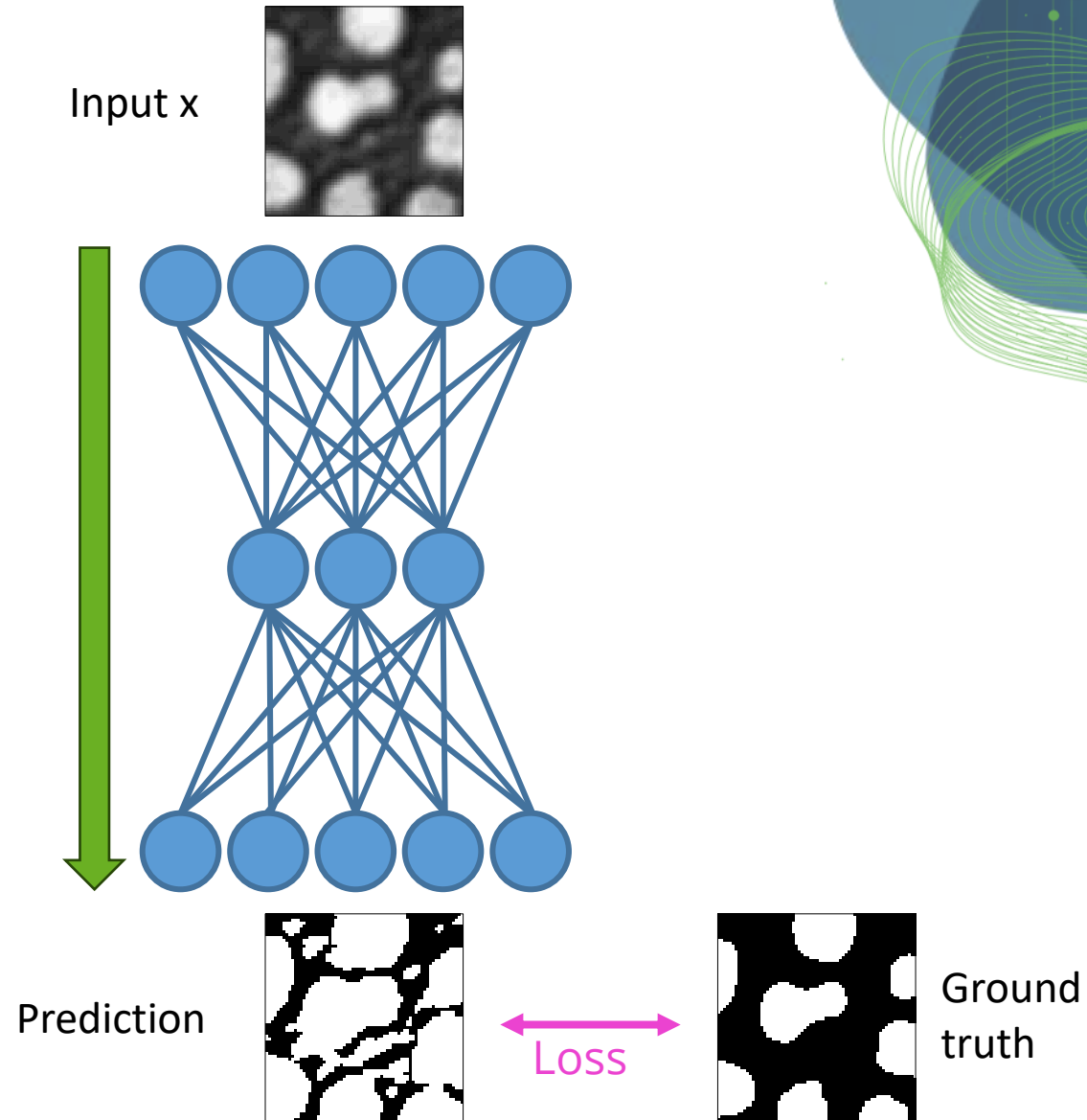
- Introduction of *non-linearity* and *activation functions* enabled what we call *deep-learning* today.



Identity		x
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1 + e^{-x}}$
Rectified linear unit (ReLU) ^[8]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max(0, x) = x \mathbf{1}_{x>0}$

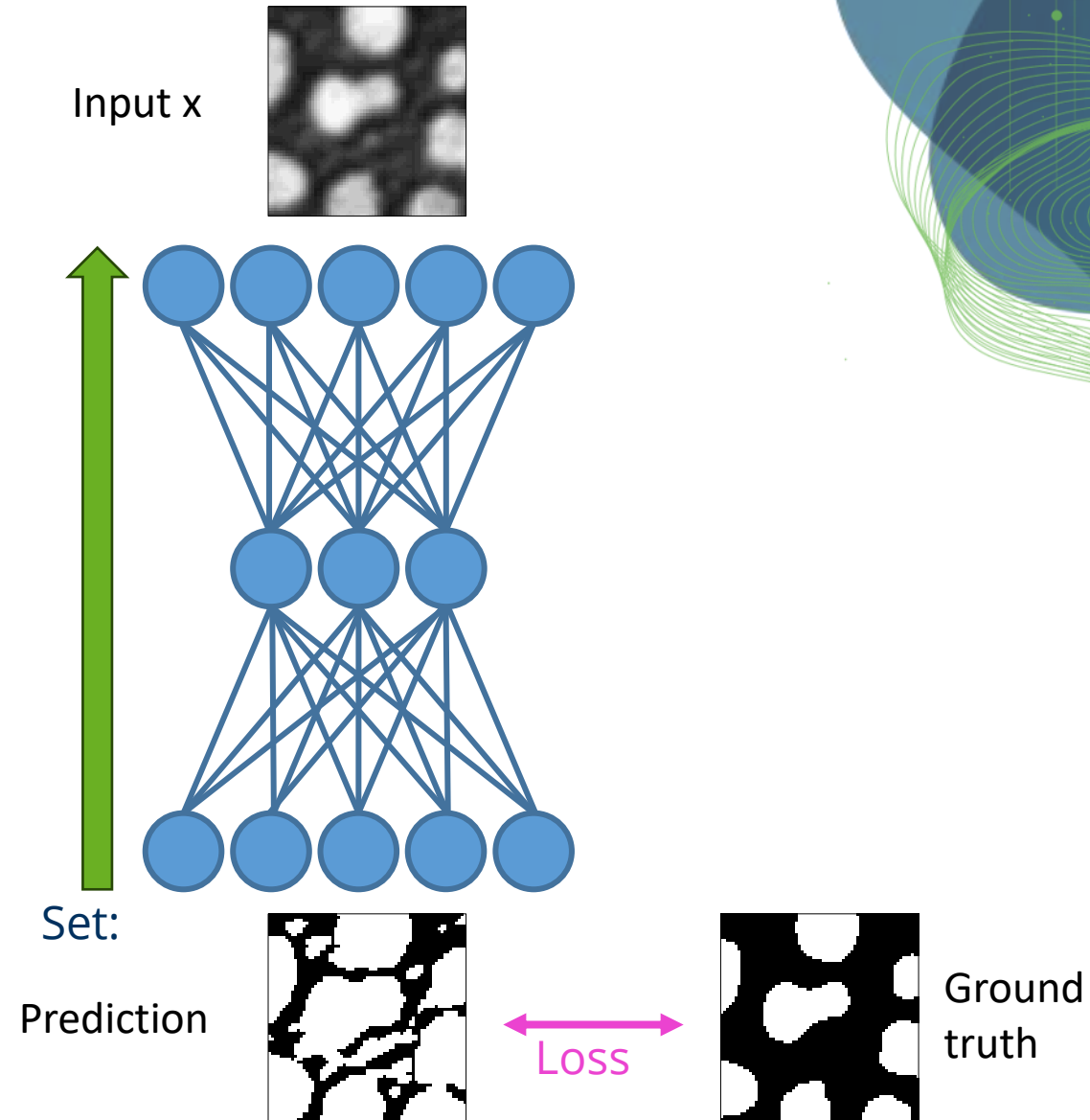
Learning: Back propagation

- Step 0: Initialize the network randomly (weights, bias)
- Step 1: **Forward pass** the input through the network, get an initial prediction
- Step 2: Compare the output with the ground truth, compute the error (loss function)
 - The **loss** function can be *freely* defined.
 - Example: mean squared error
- Step 3: Update weights



Learning: Back propagation

- Updating weights:
 - Set output to the error (per-parameter gradient)
 - **Backward-pass**: add/subtract gradients from weights, to push the network towards giving the right answer.
- Execute the same procedure for next sample
- Execute the same for multiple *epochs*



Model validation

Train dataset (e.g. 80% of the data)

- Used for training directly

Validation dataset (10% of the data)

- After every iteration see if the model overfits

Test dataset (10% of the data)

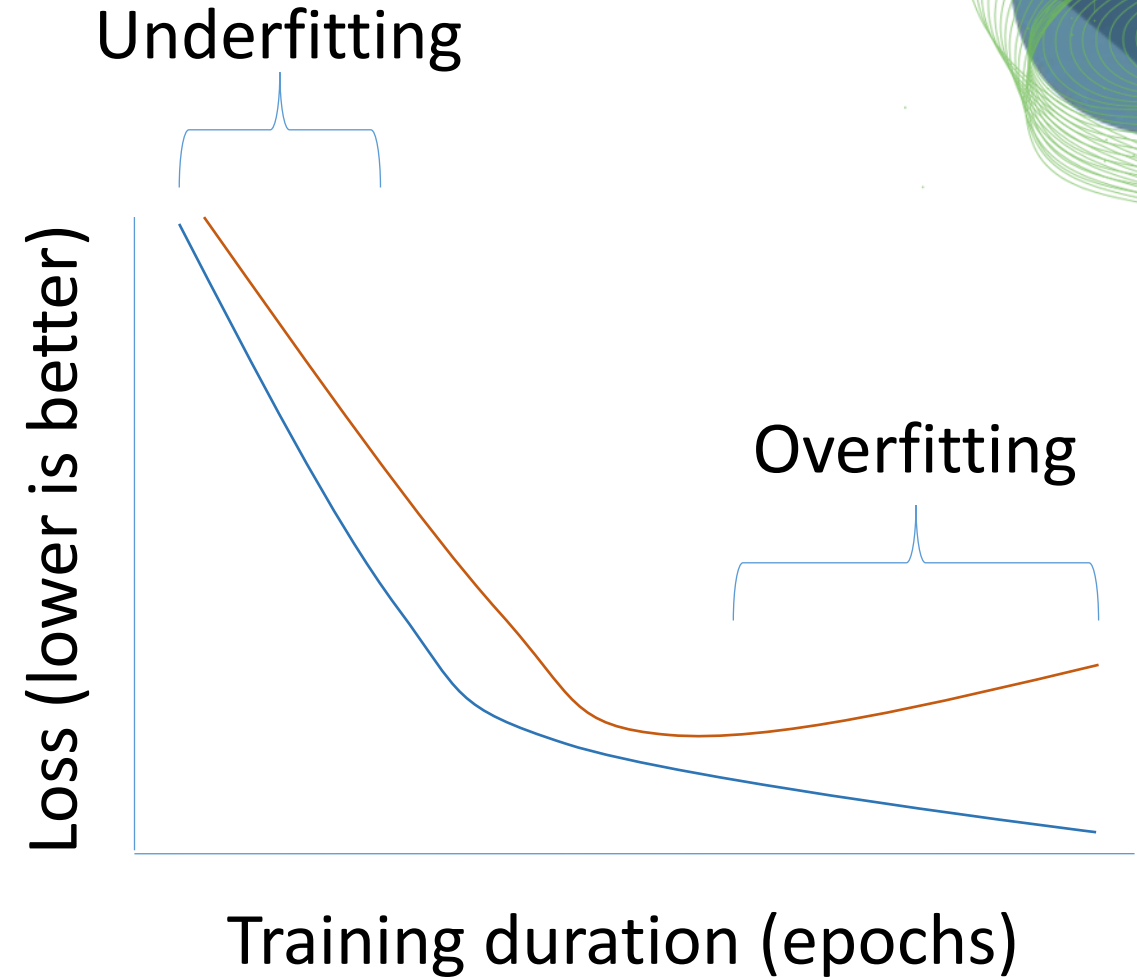
- Final evaluation after training is finished (once)

Underfitting

- A trained model that is not even able to properly process the data it was trained on

Overfitting

- A model that is able to process data it was trained on well
- It processes other data poorly



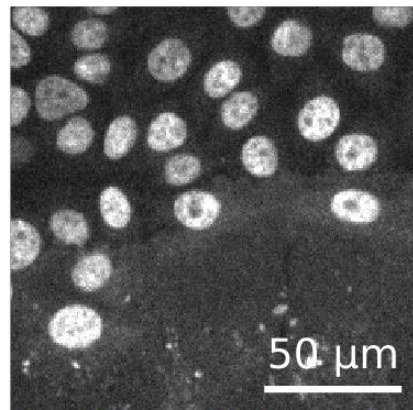
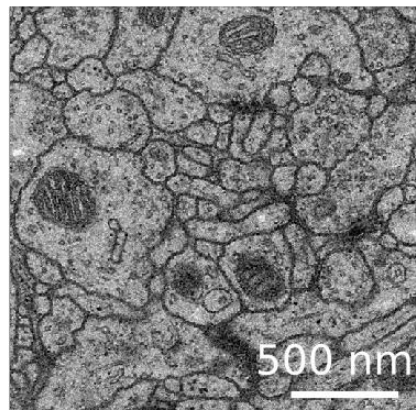
Deep Learning for Microscopy

Segmentation

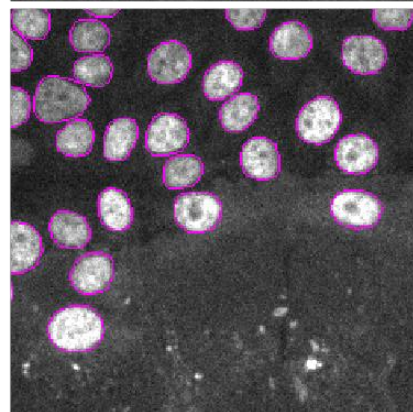
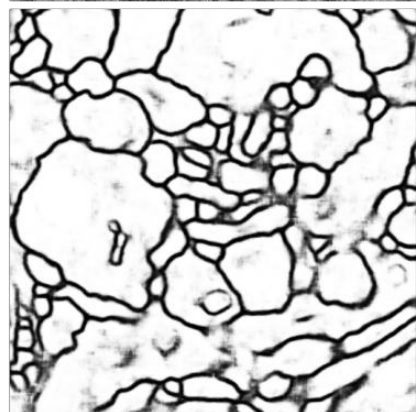
U-net

Stardist

Input



Output



Denoising

Noise2Void

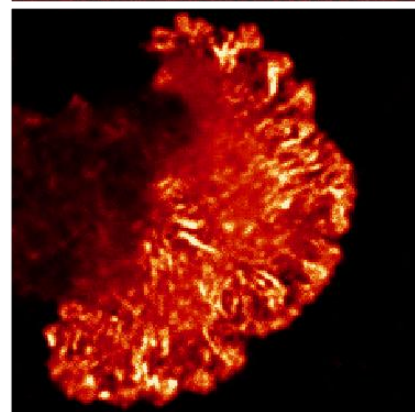
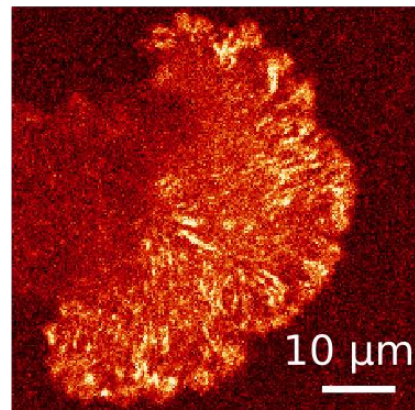
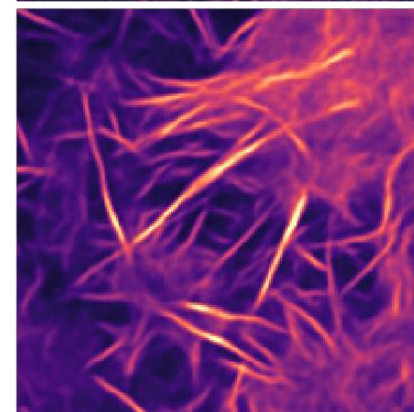
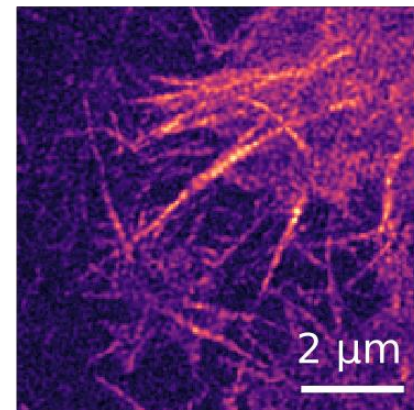


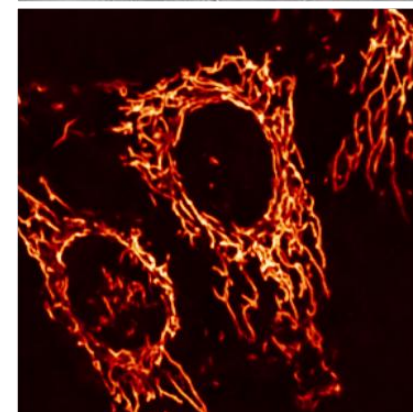
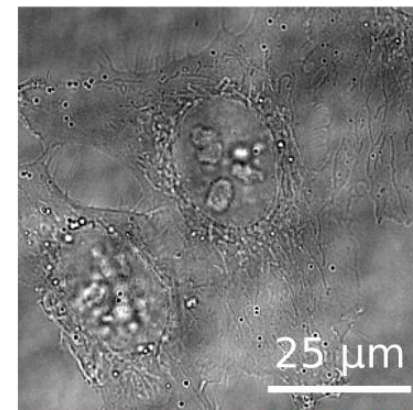
Image restoration

CARE

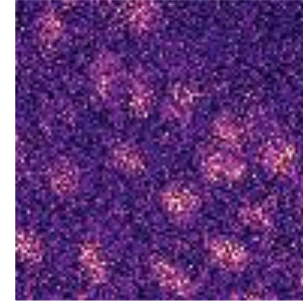
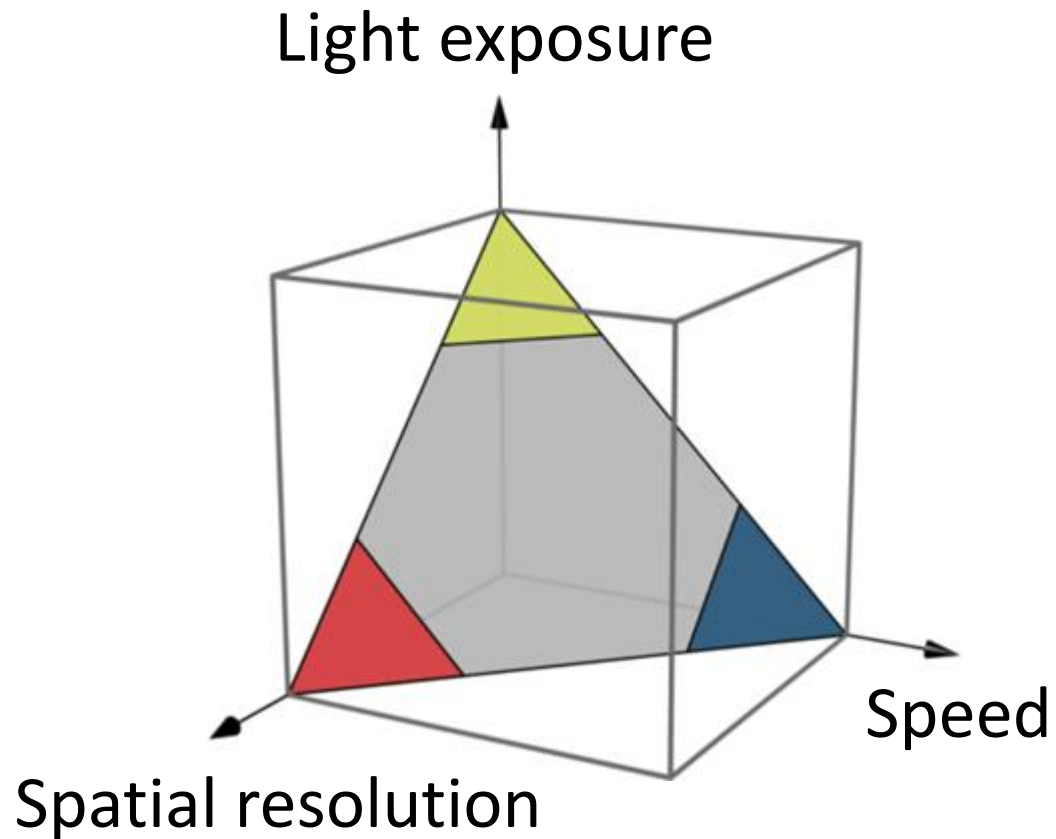


Artificial labelling

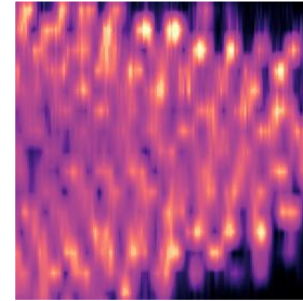
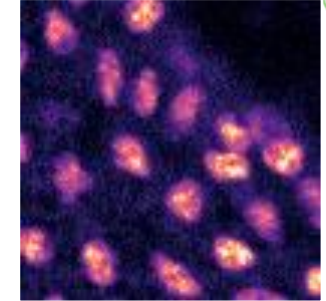
fnet



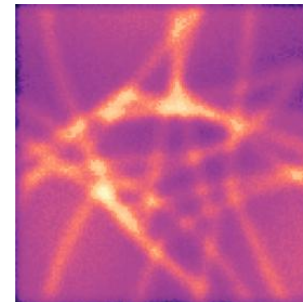
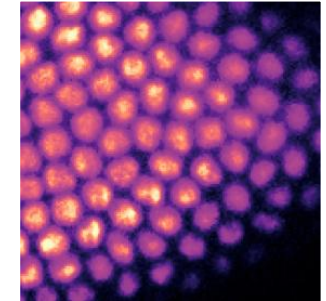
Trade-offs in live-cell imaging



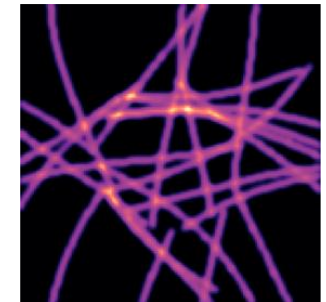
Light Exposure



Speed

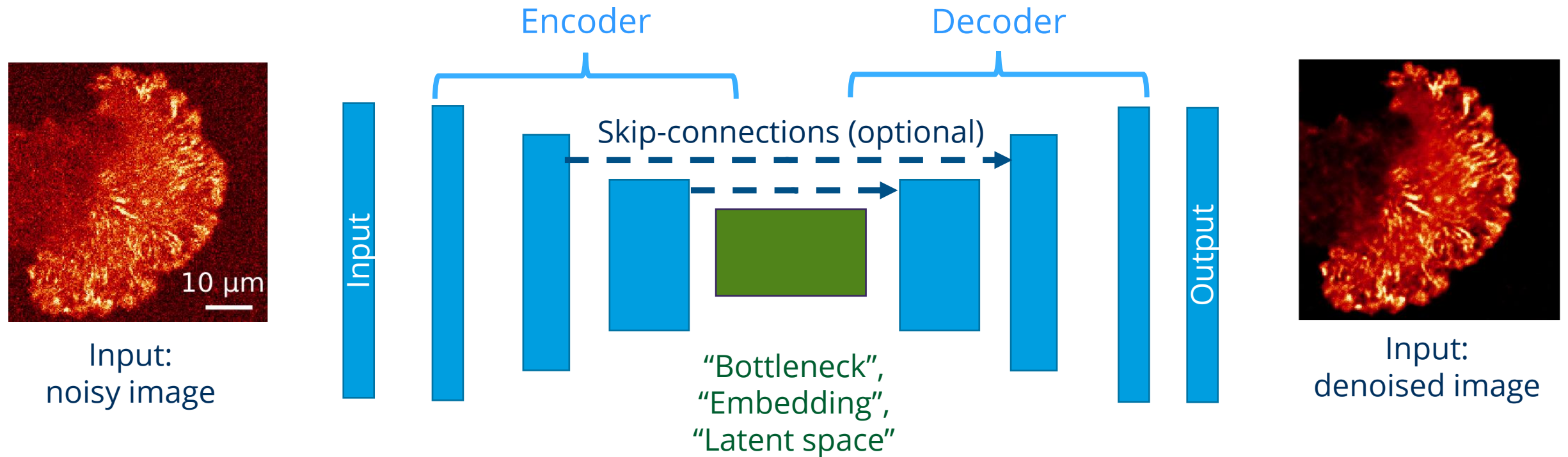


Spatial Resolution



Traditional architecture: Encoder-Decoder Networks

Related: „Auto-encoder“, „Variational Auto-Encoder“, „U-Net“



Traditional architecture: Encoder-Decoder Networks

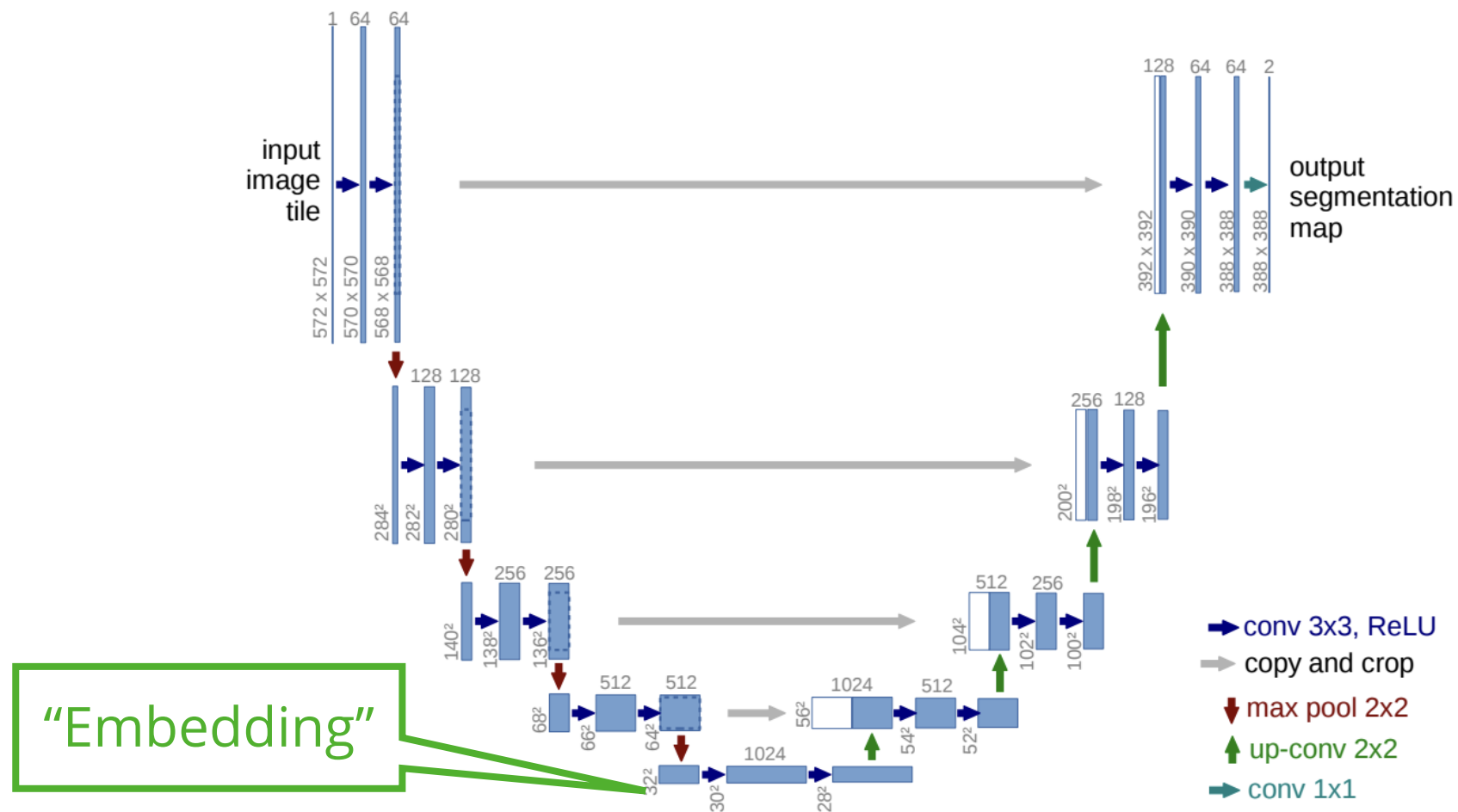


Image denoising: CARE

- Content aware image restoration (CARE)
- Image acquisition of pairs of images: A high-quality and a low-quality image.
- Problem: shot noise, biology moves!

Pair of
images
required!

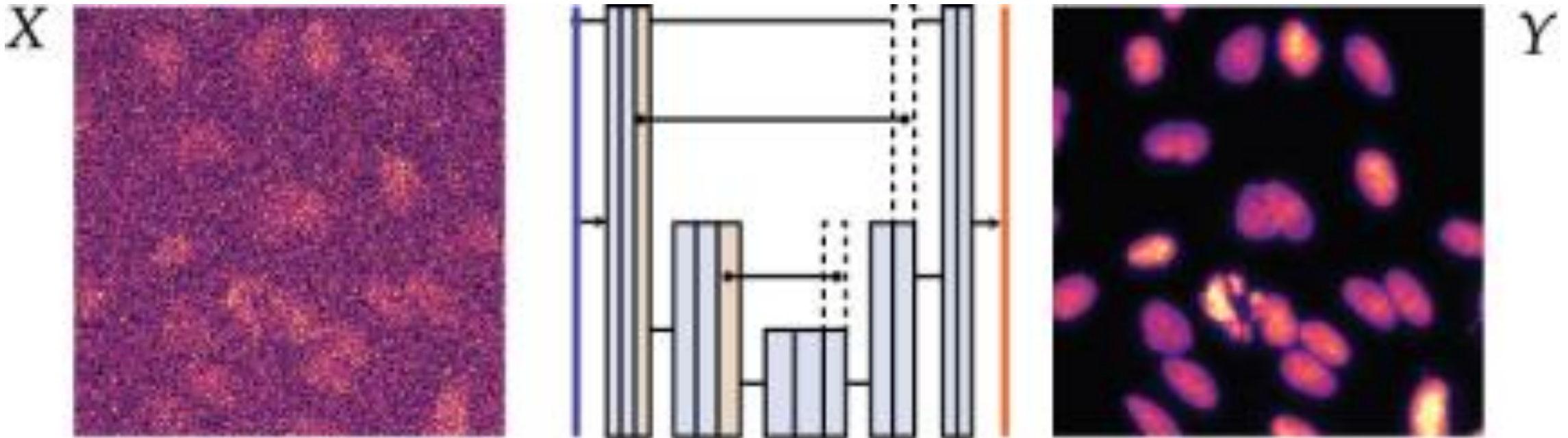
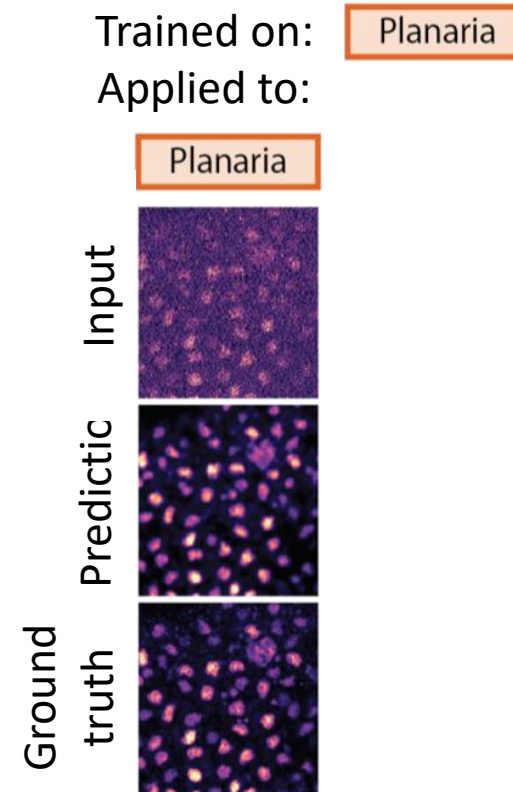


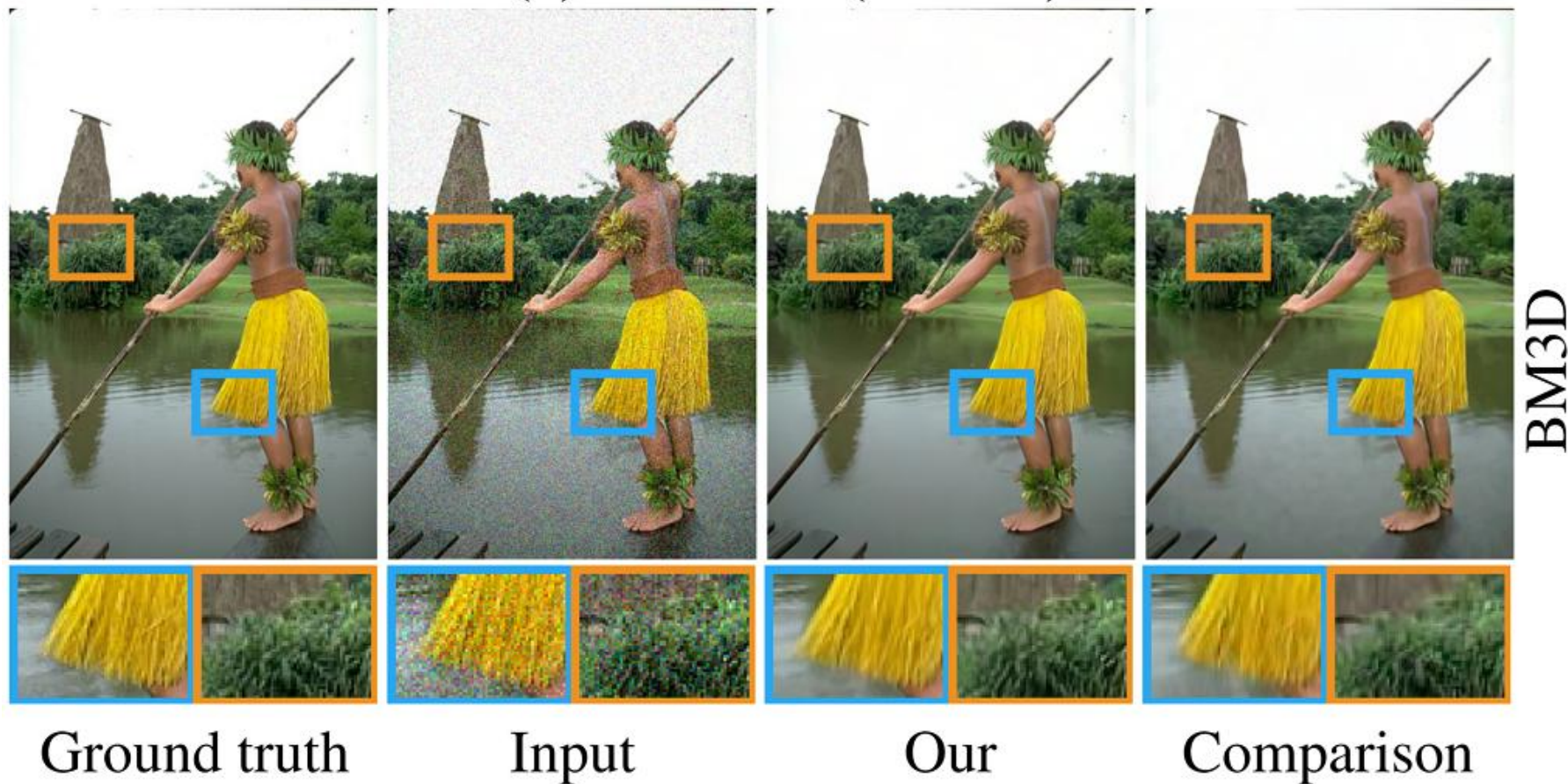
Image denoising: CARE

- Important to use on the same conditions/structures/staining that the networks were trained on!



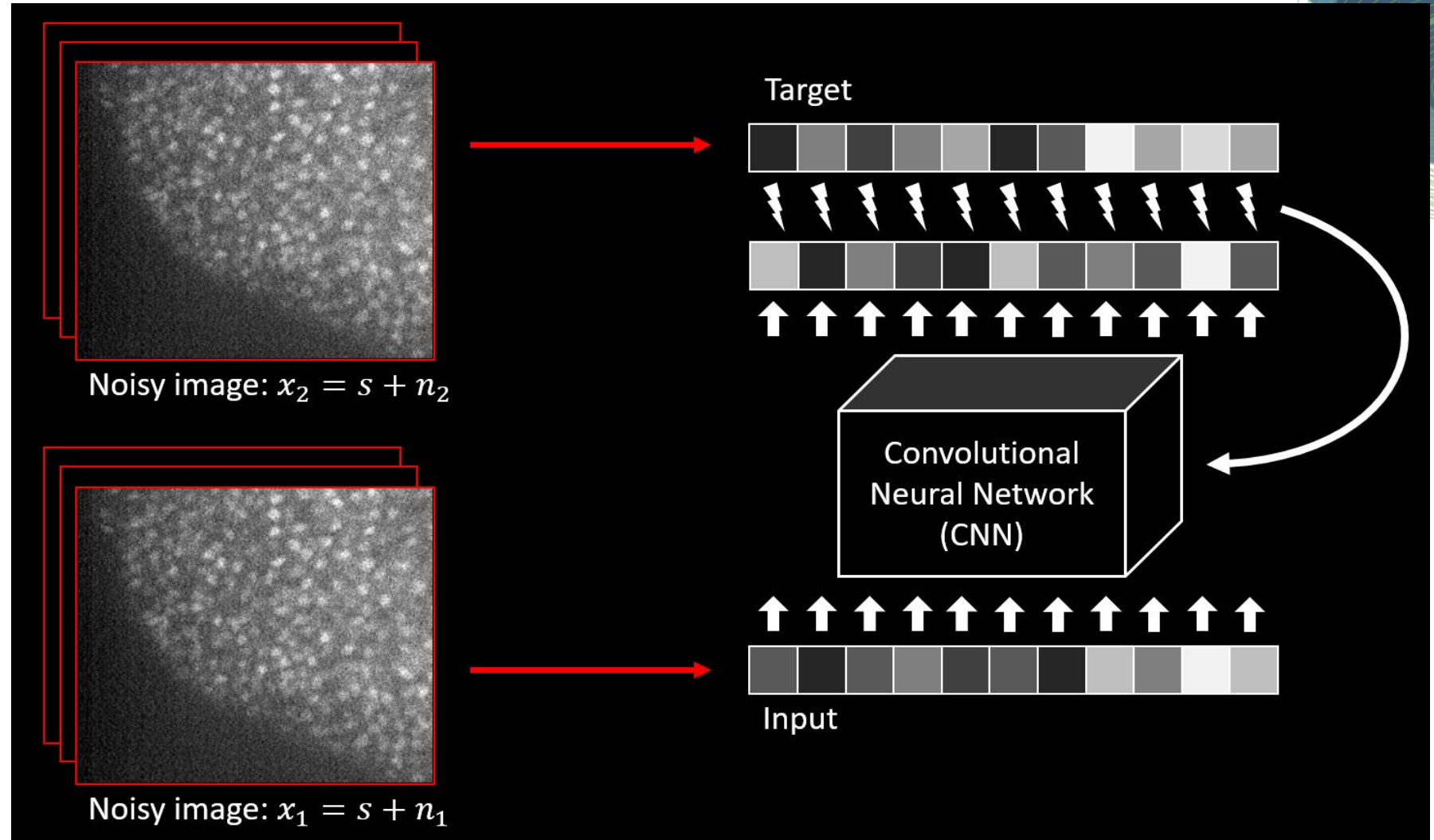
Noise2noise

(a) Gaussian ($\sigma = 25$)



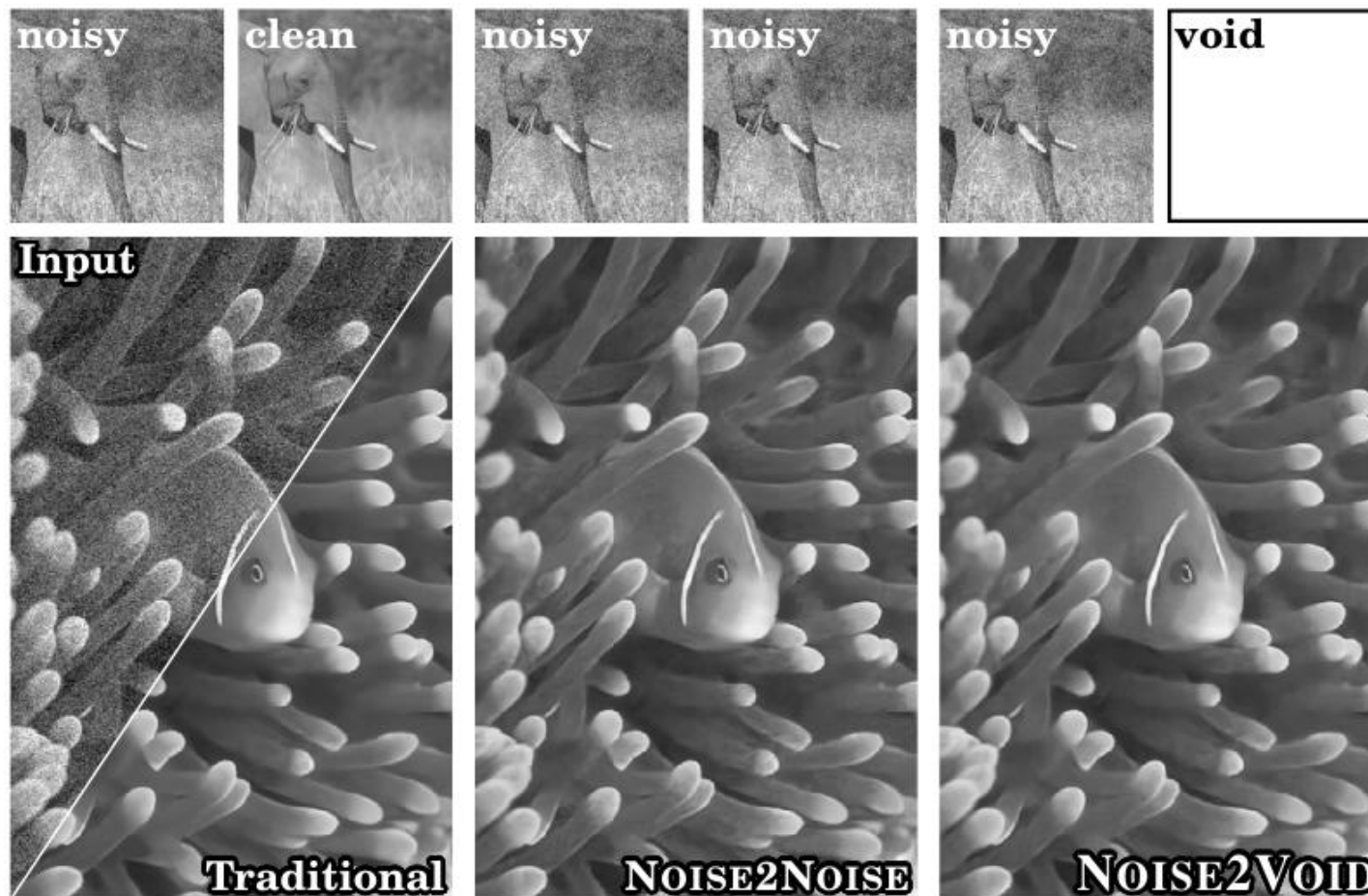
Noise2noise

- by Nvidia (Lehtinen 2018)
<https://arxiv.org/pdf/1803.04189.pdf>



Noise2void

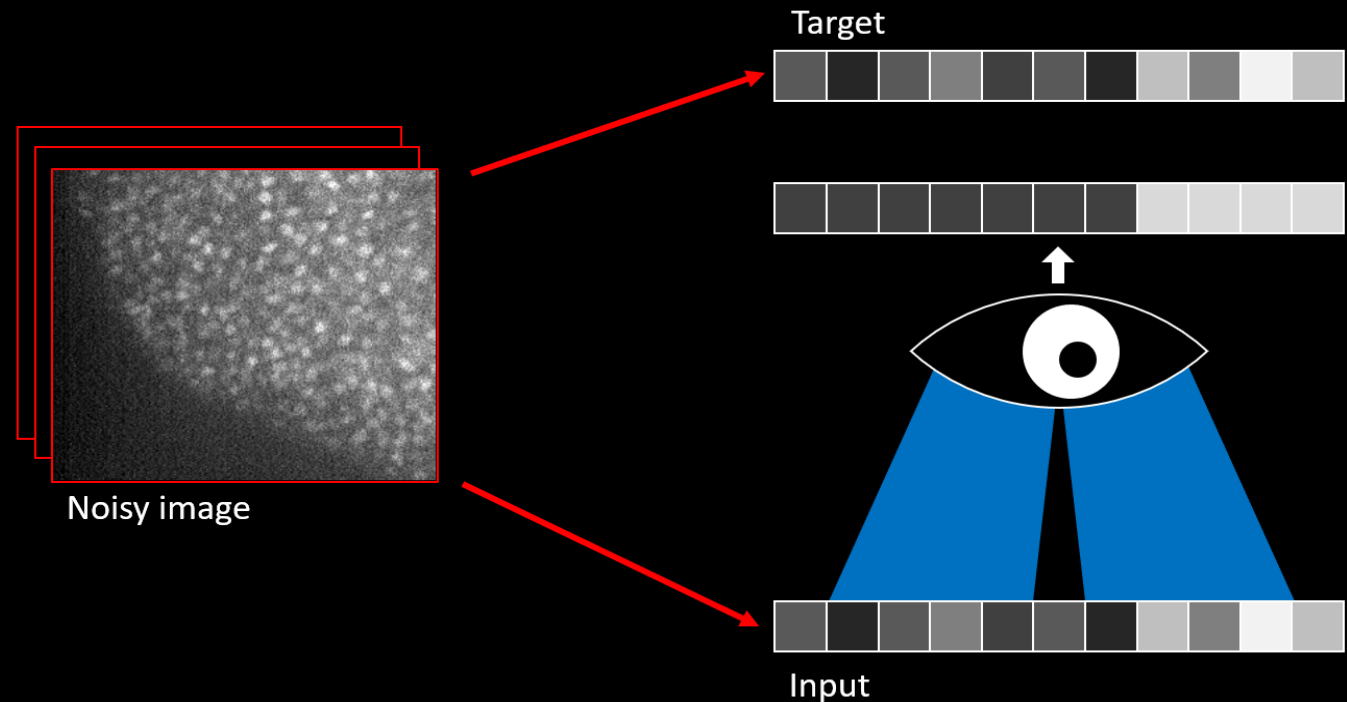
Image denoising without image pairs



Noise2void

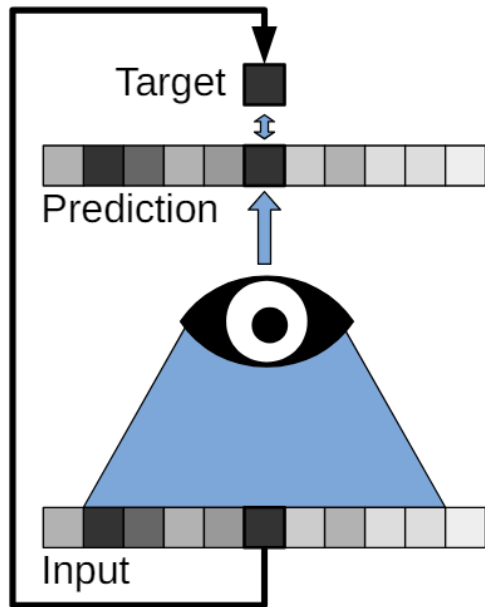
- Noise2void: Krull et al (2019)
<https://arxiv.org/abs/1811.10980>
- Noise2self: Batson and Royer (2019)
<https://arxiv.org/abs/1901.11365>

Noise2Void - Blind Spot Network

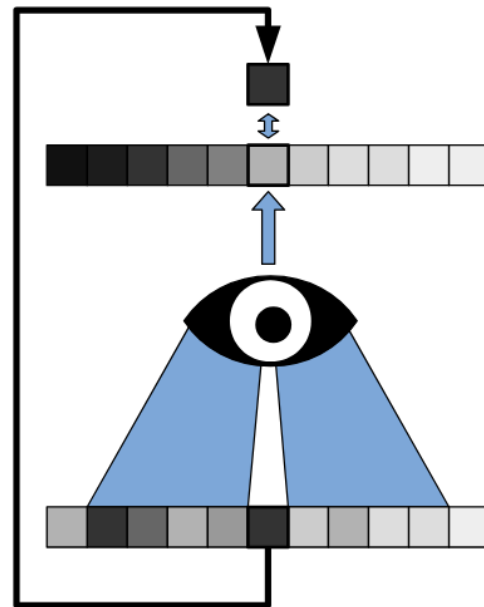


Noise2void

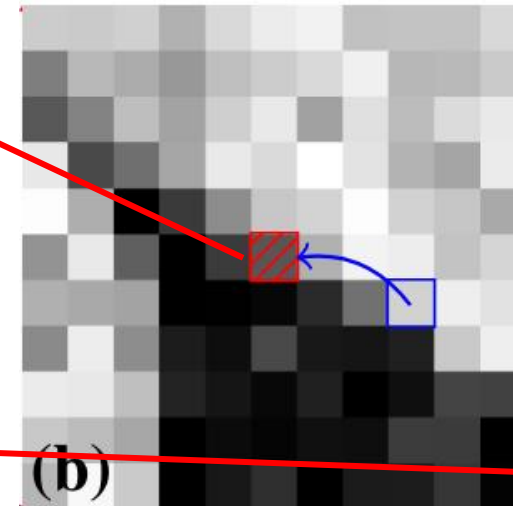
„Blind-spot-network“



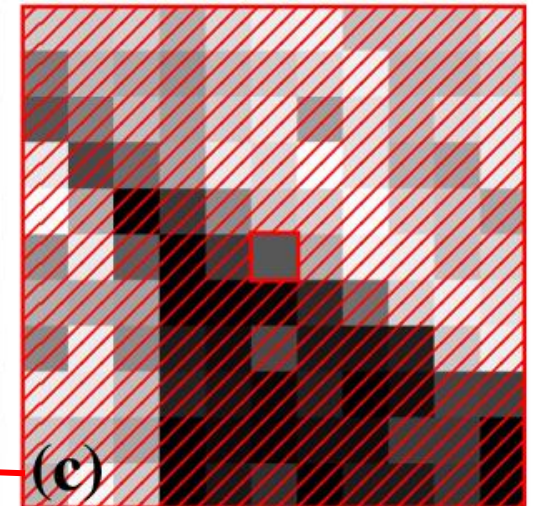
(a)



(b)



(b)



(c)

Noise2void

Napari-plugin

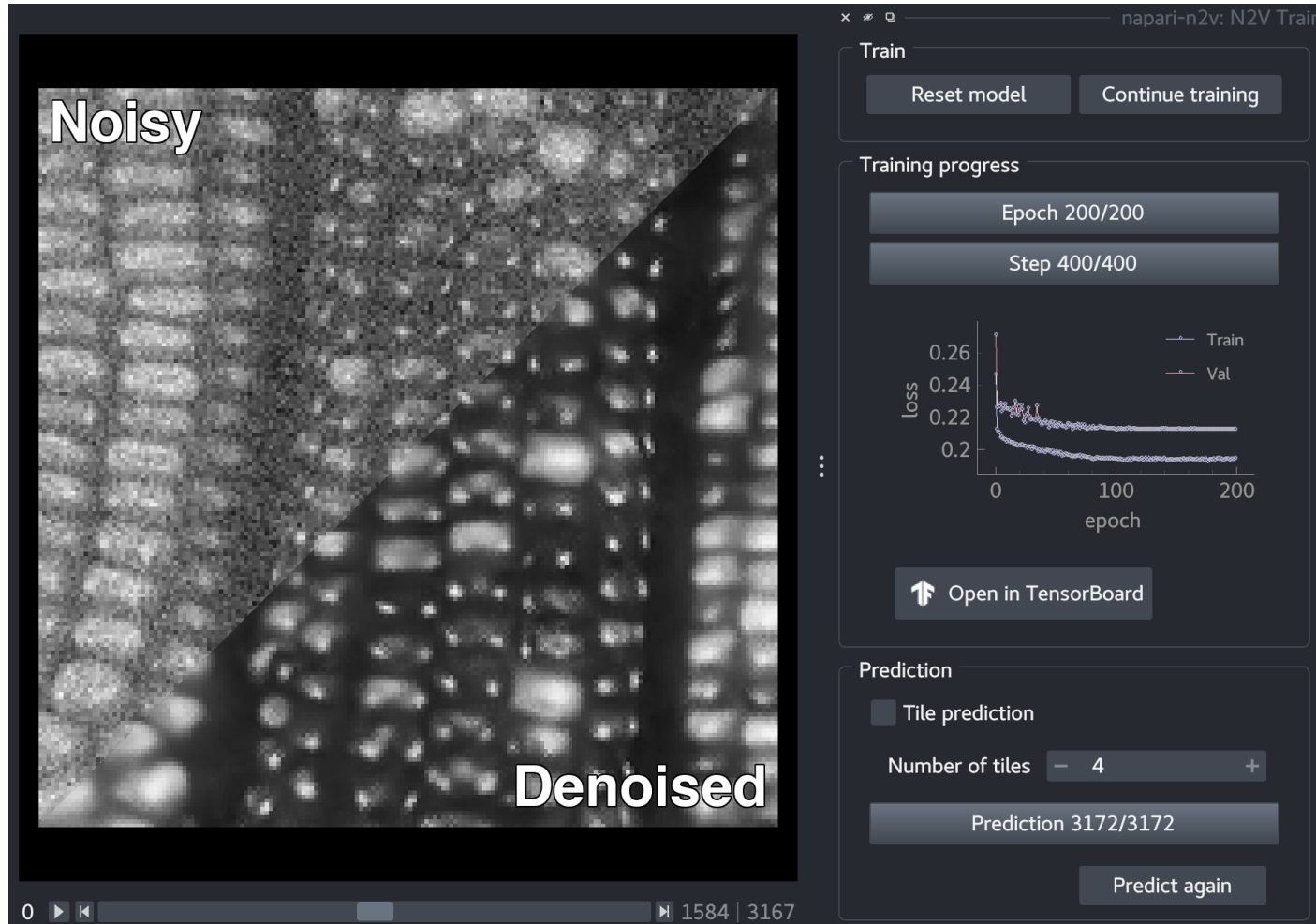
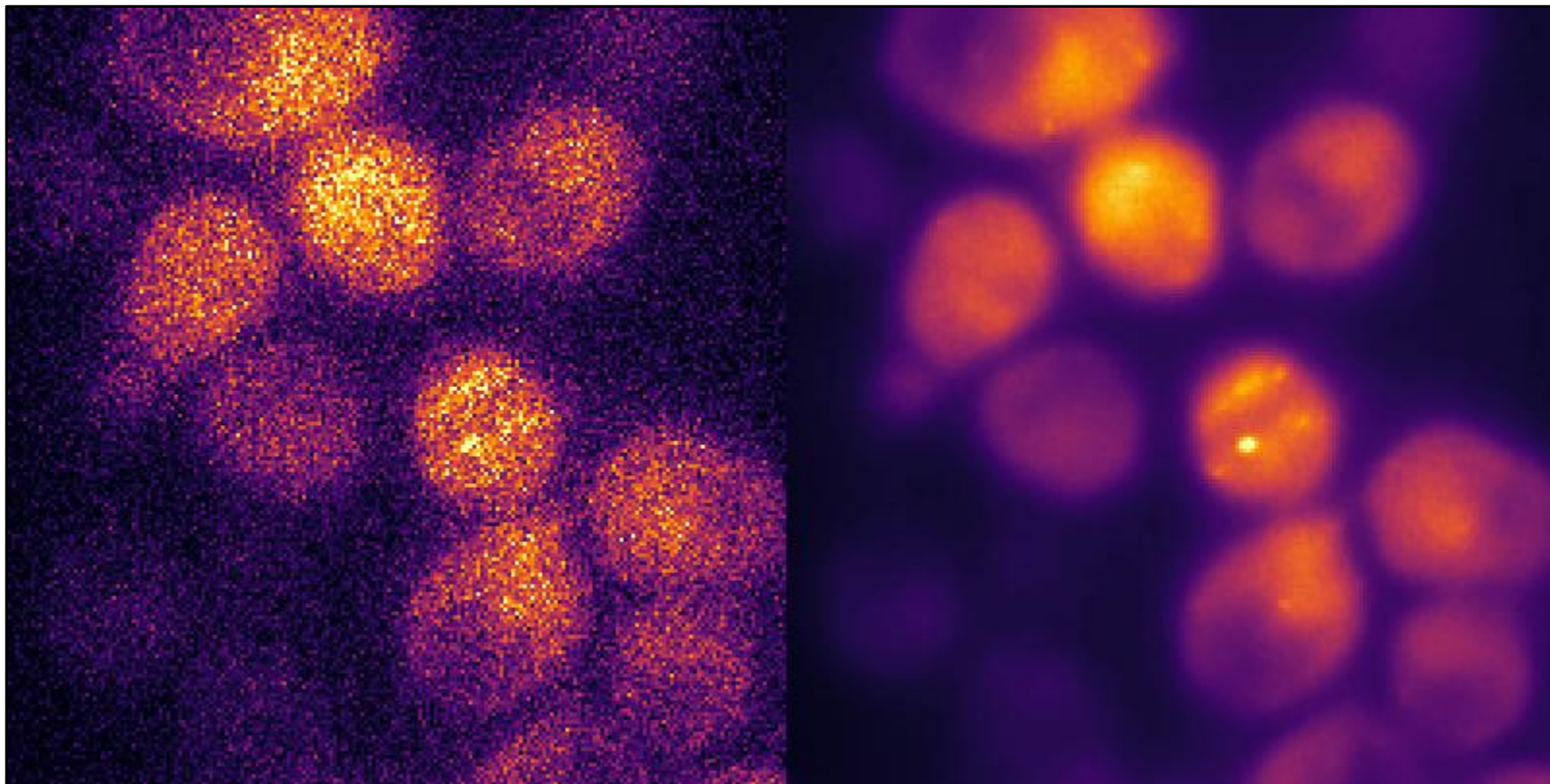


Image denoising

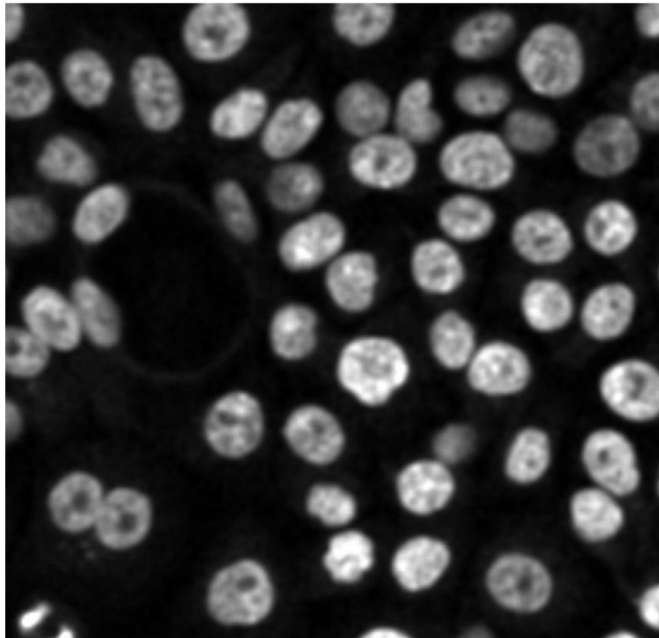
- Noise2Void



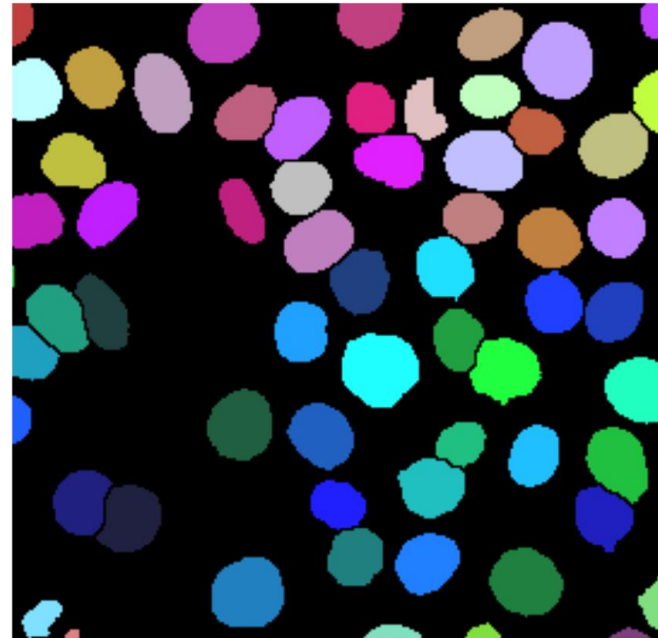
StarDist: Nuclei segmentation

Advanced algorithms are necessary when nuclei become too dense.

Image of nuclei



Corresponding masks



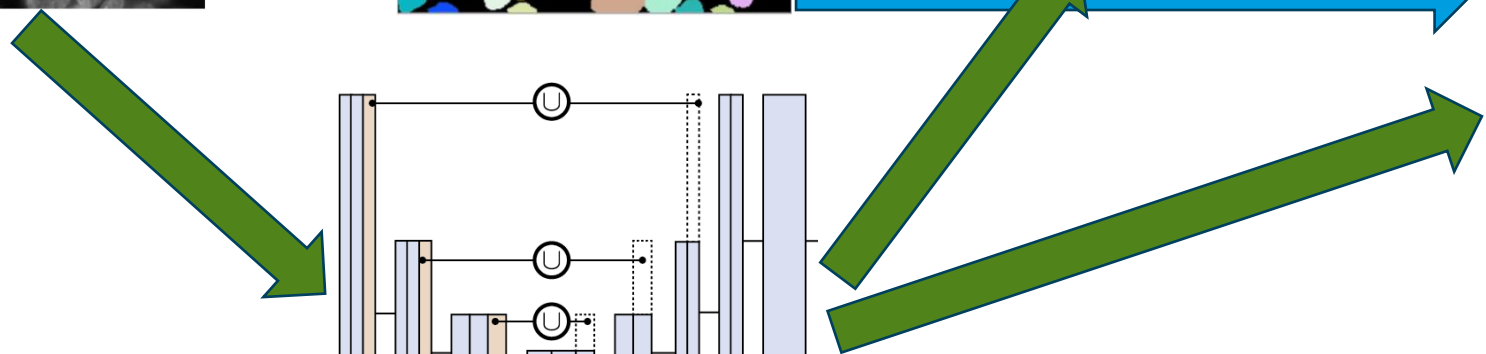
-



Class. Image proc.



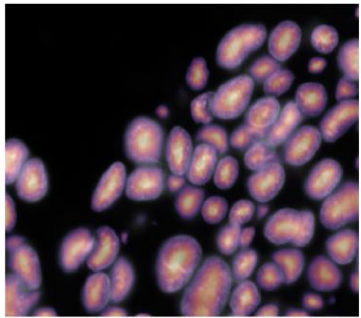
Deep learning



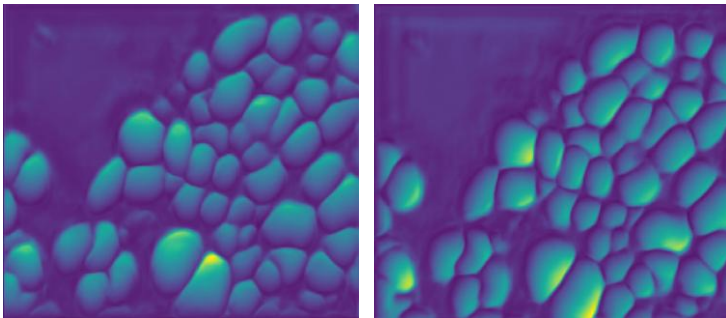
StarDist

- Prediction of probable object centers + polygon outlines
- Non-maximum-suppression of less likely polygons

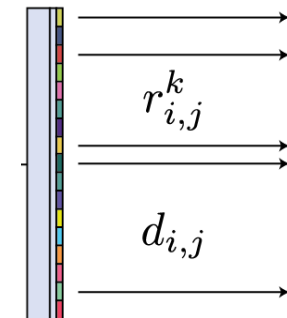
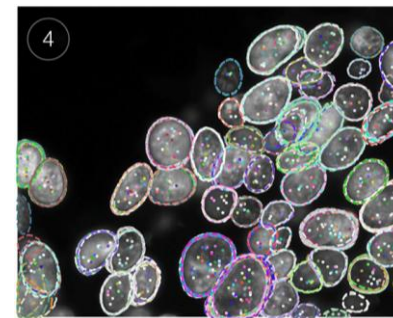
Object probabilities



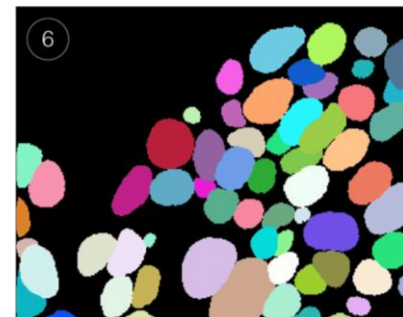
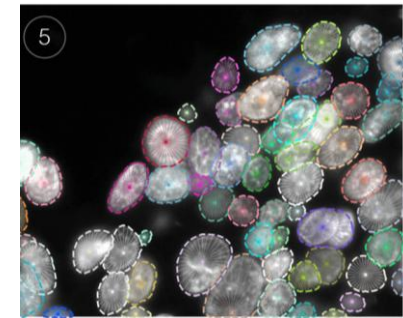
Directional distance maps



Polygon candidates

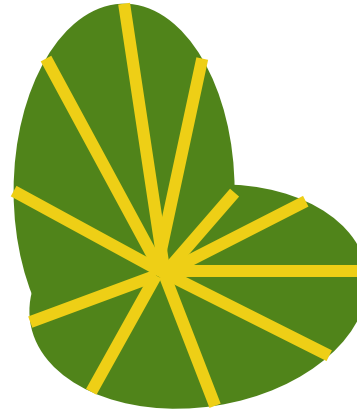
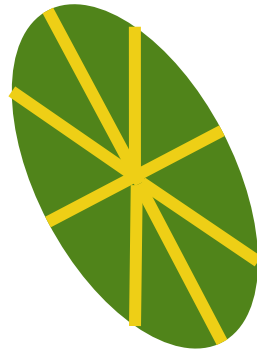
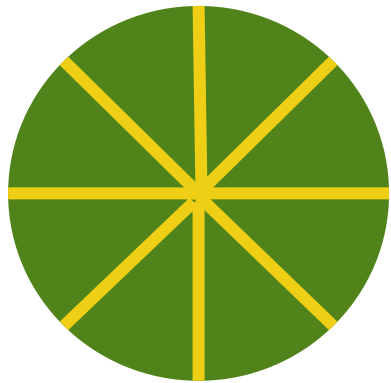


Final segmentation



StarDist: Limitations

Star-convex shapes!



Good for nuclei, bad for cells.



Quiz

Which shape features may be compromised if I use StarDist for segmentation?

StarDist: Python

Initialize model / download pretrained model

```
[3]: # creates a pretrained model
model = StarDist2D.from_pretrained('2D_versatile_fluo')

Found model '2D_versatile_fluo' for 'StarDist2D'.
Loading network weights from 'weights_best.h5'.
Loading thresholds from 'thresholds.json'.
Using default values: prob_thresh=0.479071, nms_thresh=0.3.
```

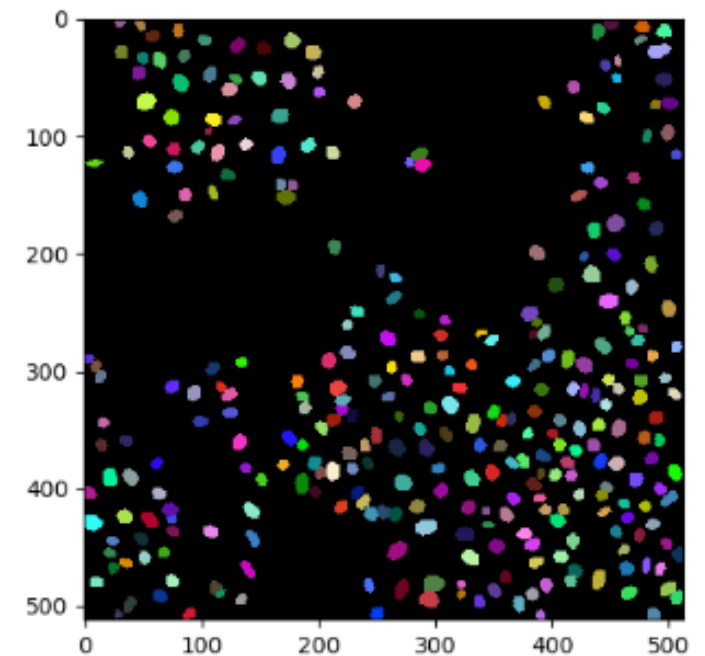
Normalize intensity to range [0, 1]

```
[4]: axis_norm = (0,1)
image = normalize(image, 1,99.8, axis=axis_norm)
```

Apply model

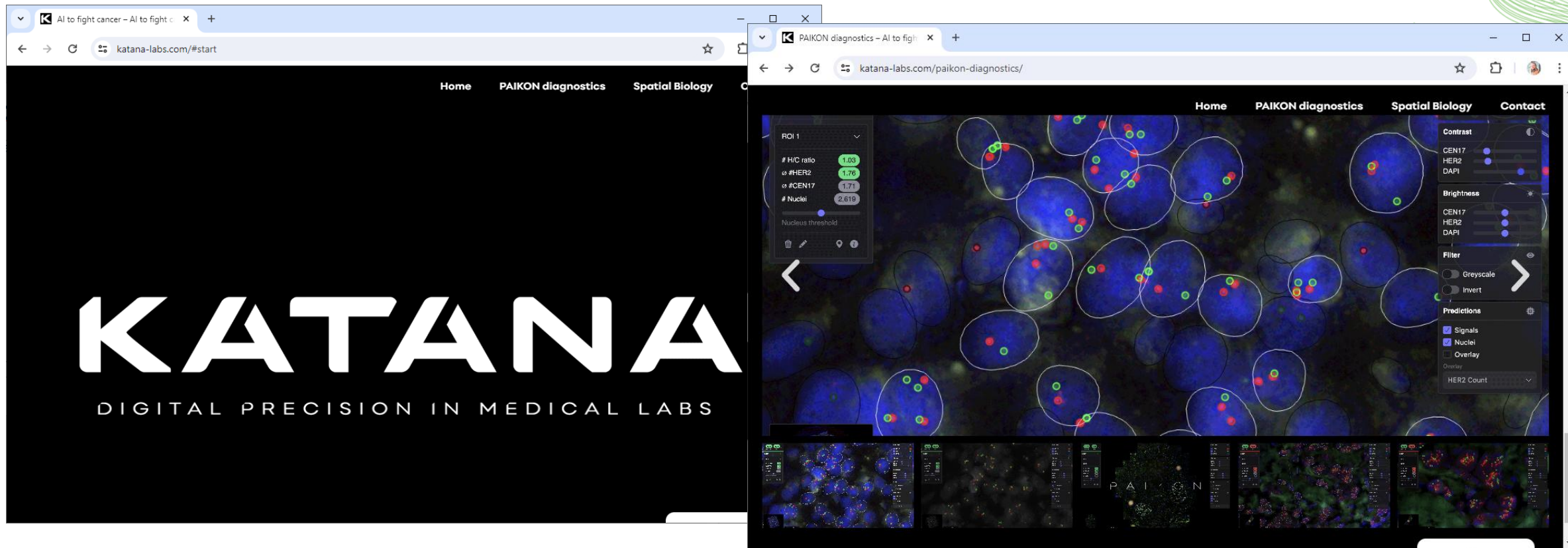
```
[5]: labels, details = model.predict_instances(image)
stackview.insight(labels)
```

[5]:



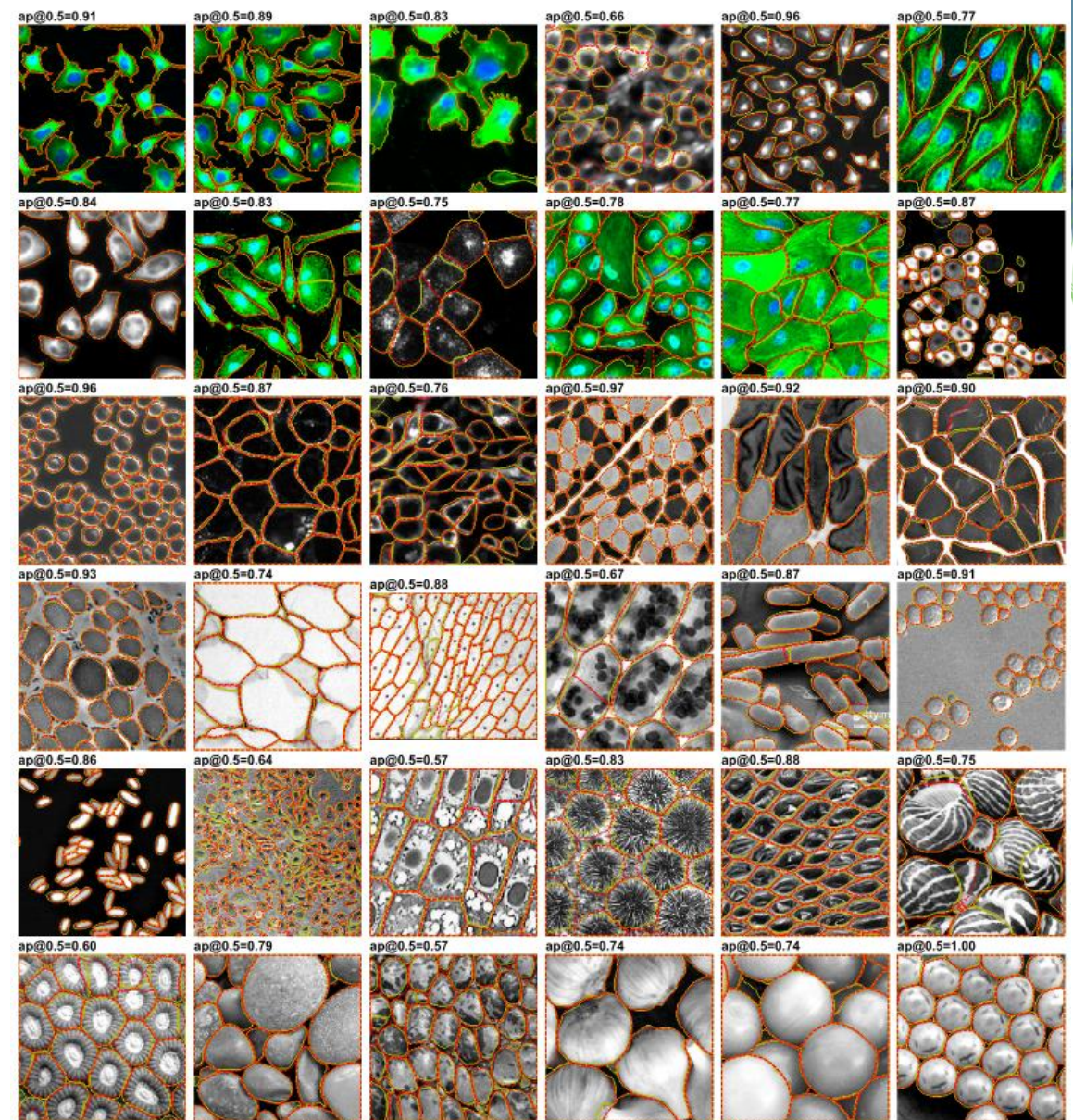
StarDist: Business model

Serving clinical scientists using software based on open-source code



CellPose

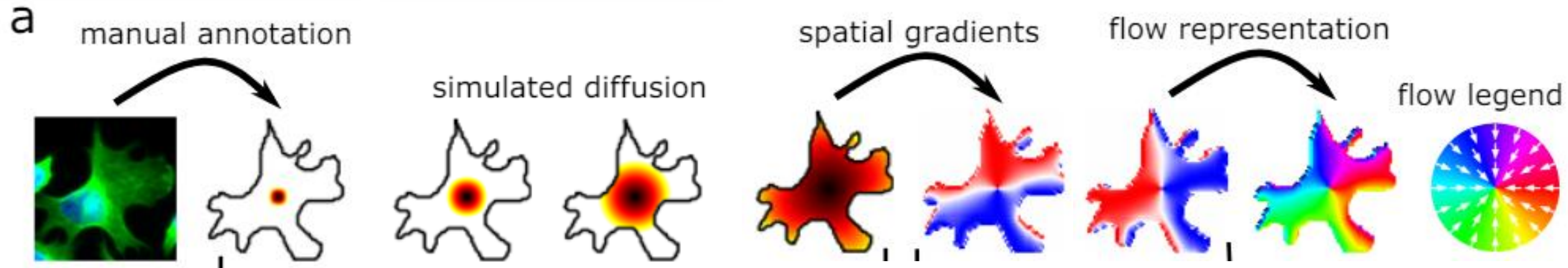
- Processing more diverse datasets and object shapes



CellPose

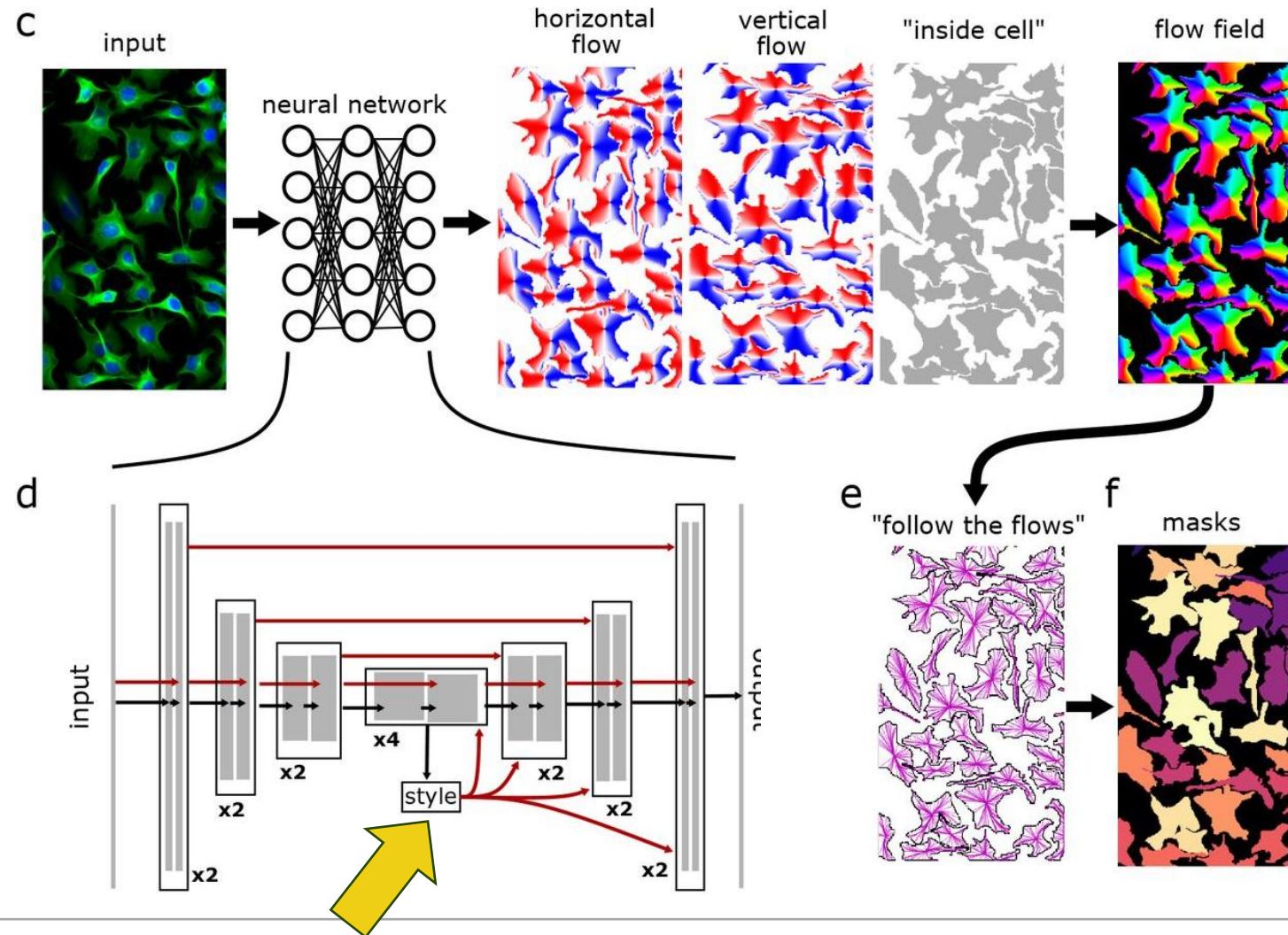
Compute „flow fields“ from images

- using classical image processing during training
- using neural networks during prediction



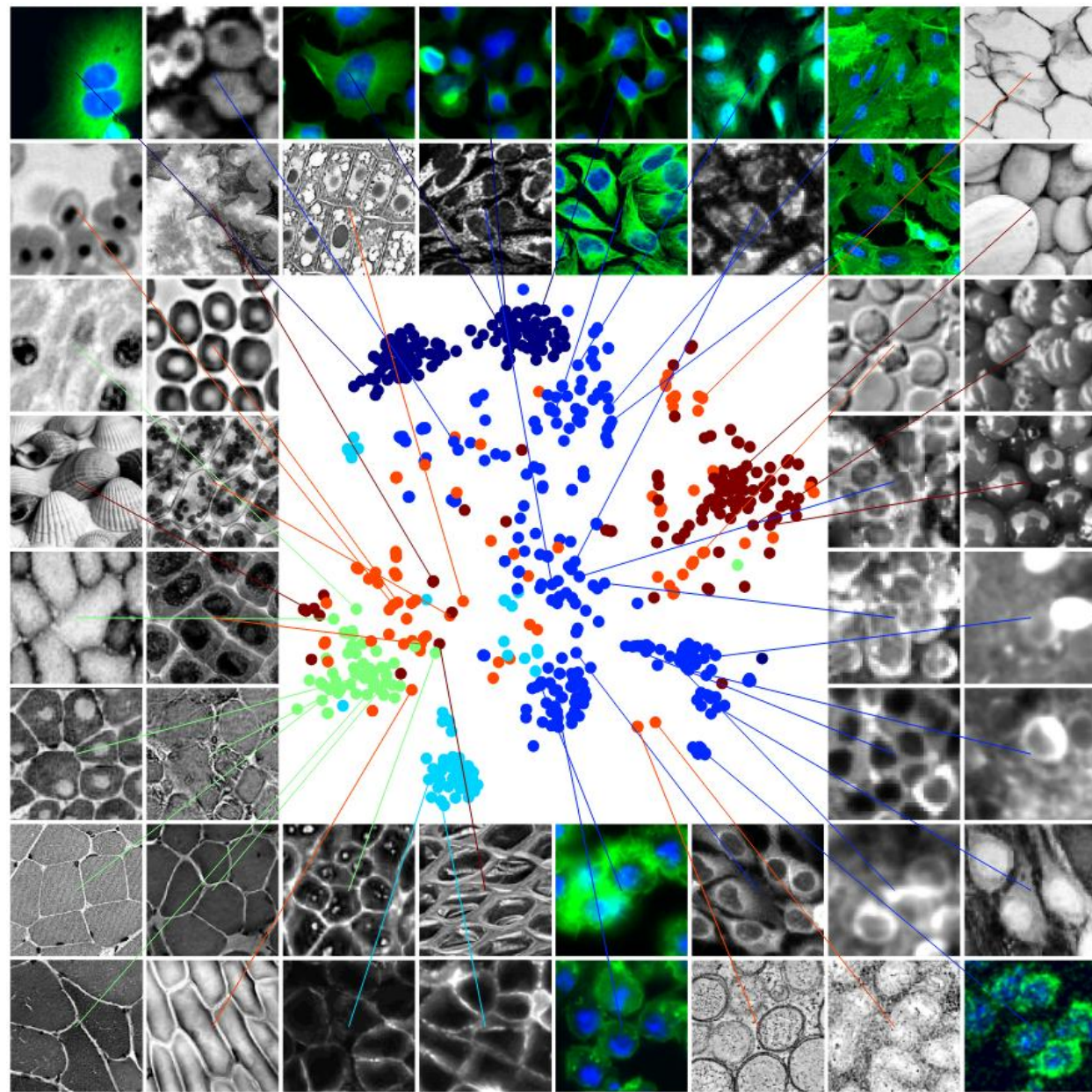
CellPose

- Cell/Nuclei – segmentation based on flow-fields
- Technically similar to Watershed, but with a deep-learning based altitude-image

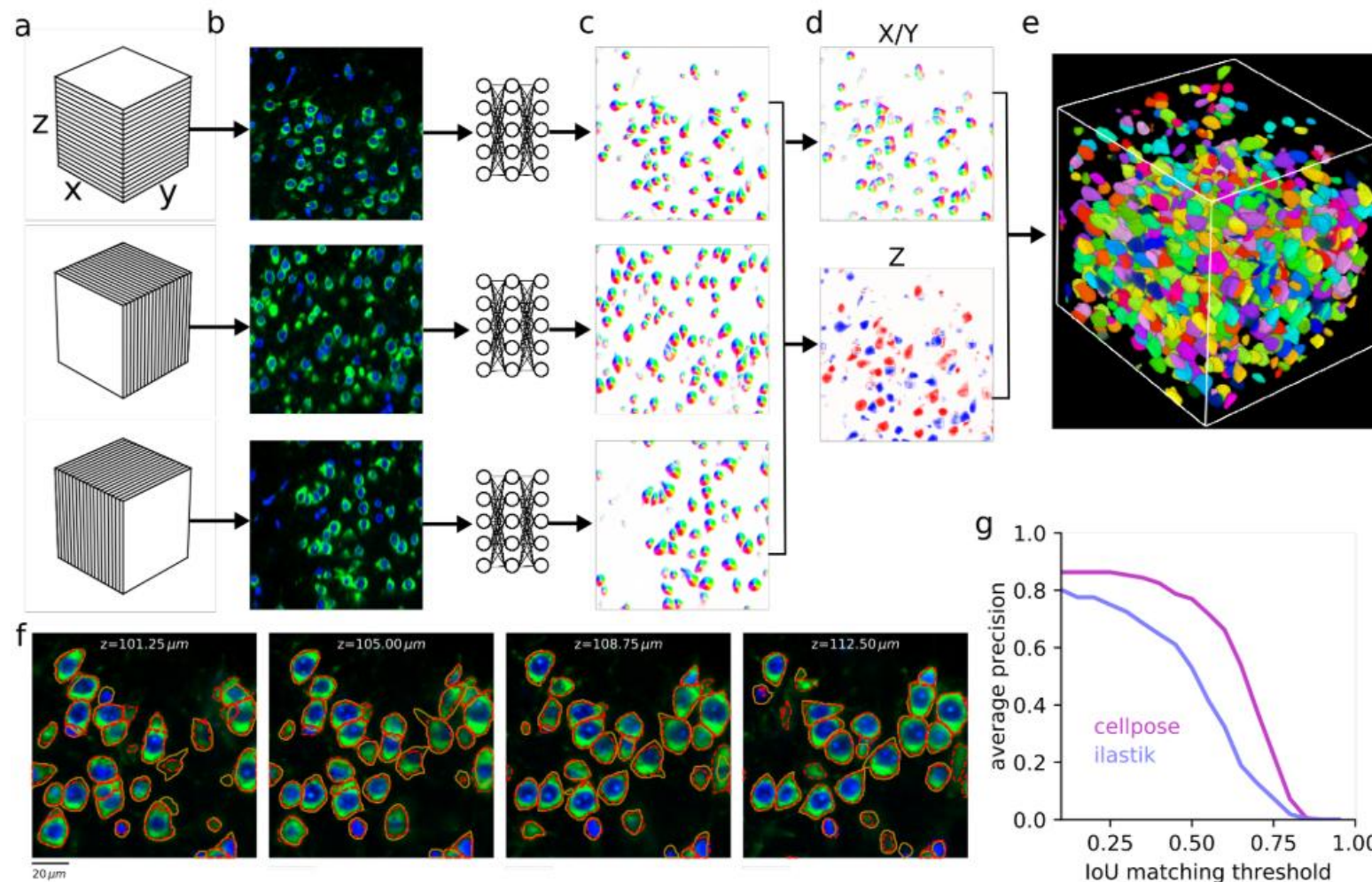


CellPose

- Image *style* is a parameter determined before prediction to guide segmentation.

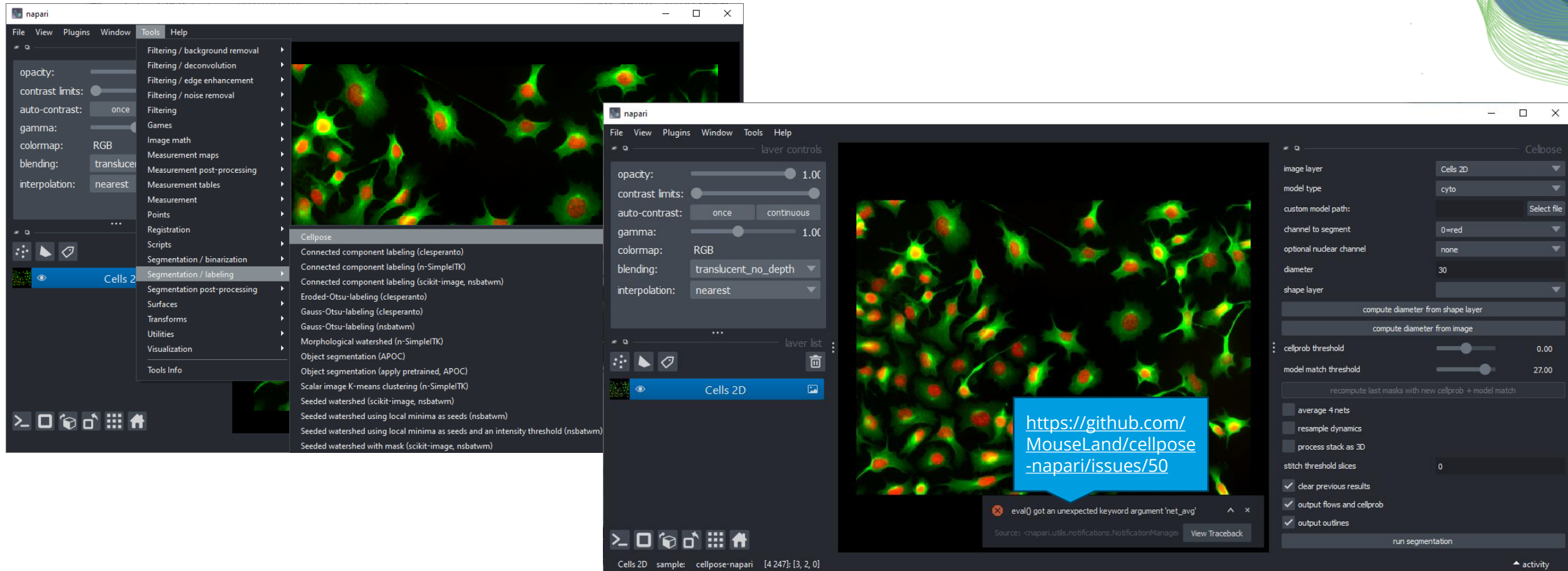


Cellpose 3D



Cellpose in Napari

conda install cellpose-napari



Cellpose in Python

Initialize a pretrained model

```
[3]: model = cellpose.models.Cellpose(gpu=False, model_type='nuclei')
```

List available models

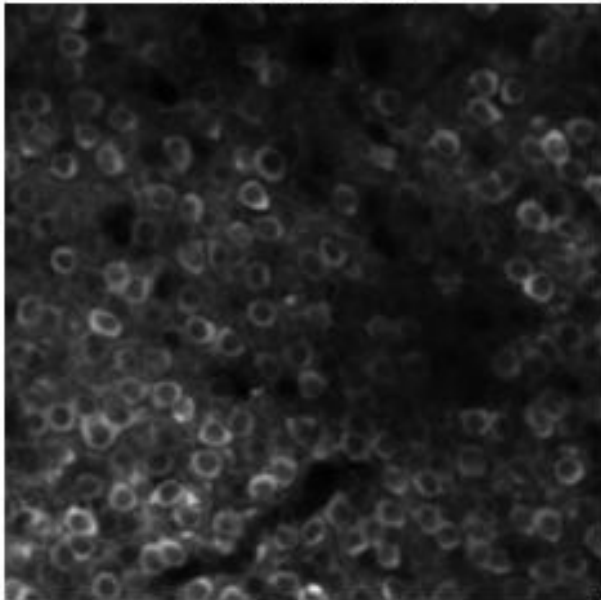
```
[8]: cellpose.models.MODEL_NAMES
```

```
[8]: ['cyto3',  
      'nuclei',  
      'cyto2_cp3',  
      'tissuenet_cp3',  
      'livecell_cp3',  
      'yeast_PhC_cp3',  
      'yeast_BF_cp3',  
      'bact_phase_cp3',  
      'bact_fluor_cp3',  
      'deepbacs_cp3',  
      'cyto2',  
      'cyto',  
      'transformer_cp3',  
      'neurips_cellpose_default',  
      'neurips_cellpose_transformer',  
      'neurips_grayscale_cyto2']
```

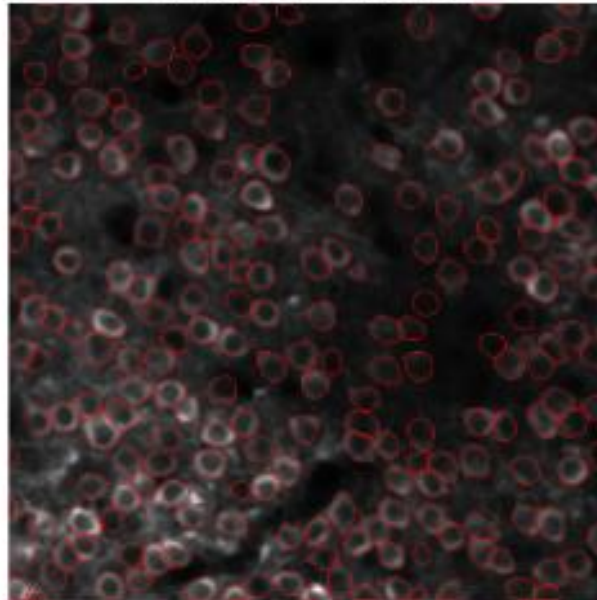
Cellpose in Python

```
[4]: channels = [0,0] # This means we are processing single-channel greyscale images.  
masks, flows, styles, diams = model.eval(image, diameter=None, channels=channels)
```

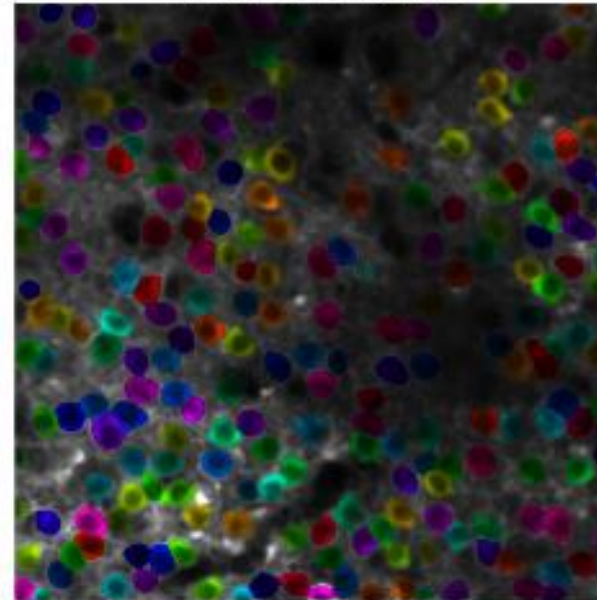
original image



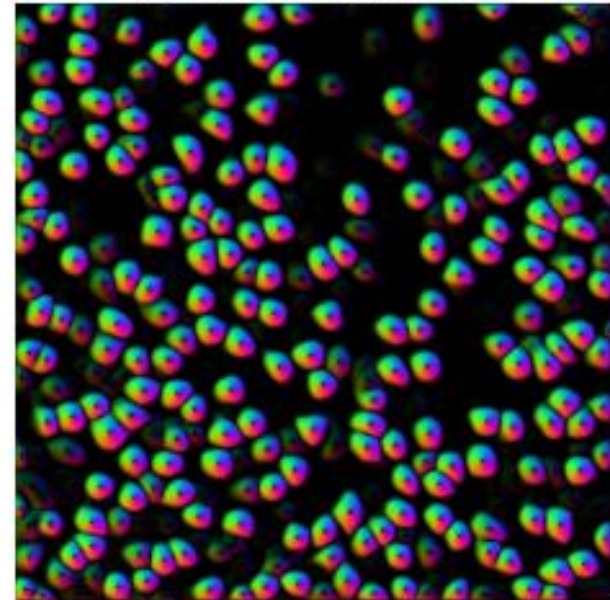
predicted outlines



predicted masks

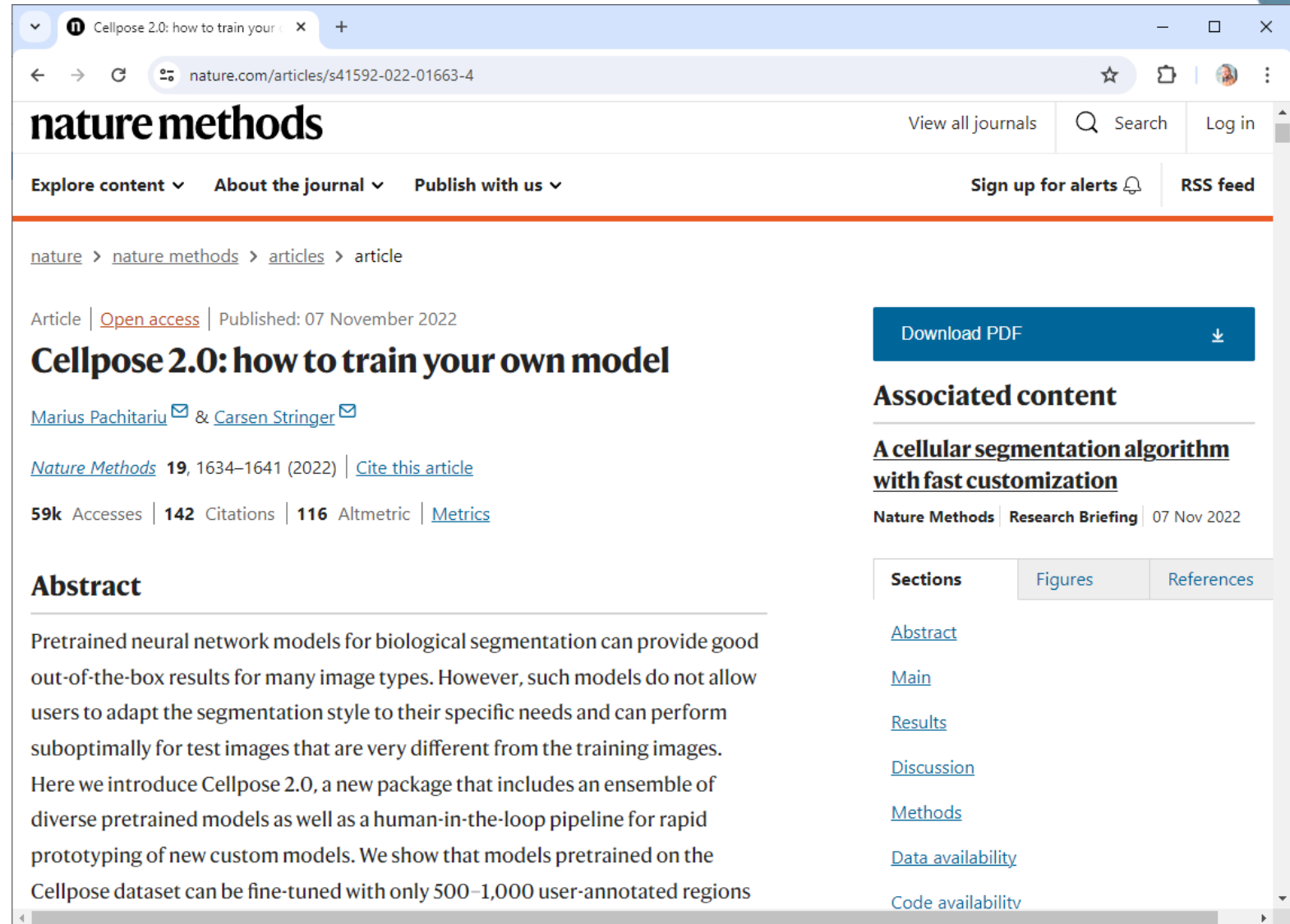


predicted cell pose



Cellpose 2

Train your own model



The screenshot shows the article page for 'Cellpose 2.0: how to train your own model' on the Nature Methods website. The browser address bar shows the URL 'nature.com/articles/s41592-022-01663-4'. The page header includes the 'nature methods' logo, navigation links like 'View all journals', 'Search', and 'Log in', and a 'Sign up for alerts' button. The article title is prominently displayed, followed by the authors 'Marius Pachitariu' and 'Carsen Stringer'. Below the title, it indicates the journal 'Nature Methods', volume '19', pages '1634–1641', and the year '(2022)'. Metrics for '59k Accesses', '142 Citations', and '116 Altmetric' are shown. The 'Abstract' section begins with the text: 'Pretrained neural network models for biological segmentation can provide good out-of-the-box results for many image types. However, such models do not allow users to adapt the segmentation style to their specific needs and can perform suboptimally for test images that are very different from the training images. Here we introduce Cellpose 2.0, a new package that includes an ensemble of diverse pretrained models as well as a human-in-the-loop pipeline for rapid prototyping of new custom models. We show that models pretrained on the Cellpose dataset can be fine-tuned with only 500–1,000 user-annotated regions'. On the right side, there is a 'Download PDF' button and a section for 'Associated content' with the title 'A cellular segmentation algorithm with fast customization'. Below this, there are tabs for 'Sections', 'Figures', and 'References', with 'Sections' currently selected. A list of links for 'Abstract', 'Main', 'Results', 'Discussion', 'Methods', 'Data availability', and 'Code availability' is provided.

Cellpose 2.0: how to train your own model

Marius Pachitariu & Carsen Stringer

Nature Methods 19, 1634–1641 (2022) | Cite this article

59k Accesses | 142 Citations | 116 Altmetric | Metrics

Abstract

Pretrained neural network models for biological segmentation can provide good out-of-the-box results for many image types. However, such models do not allow users to adapt the segmentation style to their specific needs and can perform suboptimally for test images that are very different from the training images. Here we introduce Cellpose 2.0, a new package that includes an ensemble of diverse pretrained models as well as a human-in-the-loop pipeline for rapid prototyping of new custom models. We show that models pretrained on the Cellpose dataset can be fine-tuned with only 500–1,000 user-annotated regions

Download PDF

Associated content

A cellular segmentation algorithm with fast customization

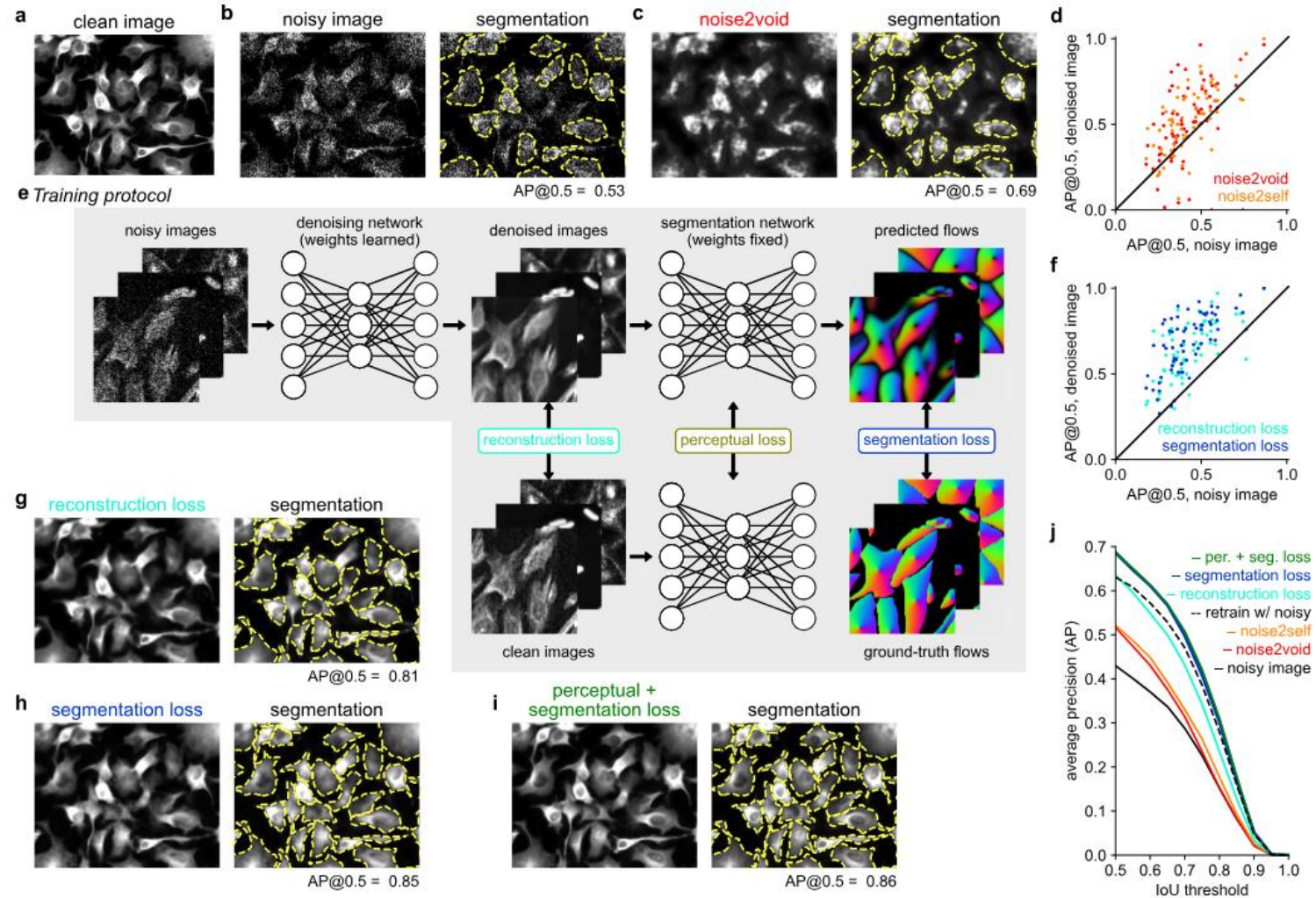
Nature Methods | Research Briefing | 07 Nov 2022

Sections | Figures | References

- Abstract
- Main
- Results
- Discussion
- Methods
- Data availability
- Code availability

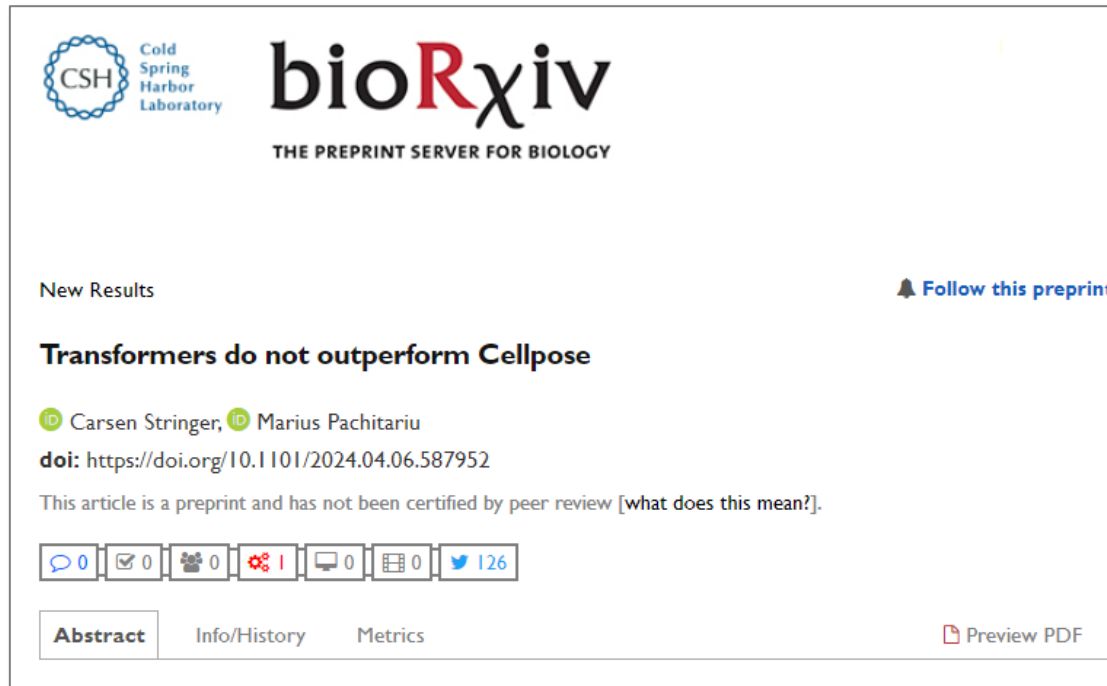
Cellpose 3

Cellpose 3 includes denoising and deblurring, to improve image segmentation quality



Cellpose

State-of-the-art despite the rise of transformers



The screenshot shows the bioRxiv preprint interface. At the top left is the CSH Cold Spring Harbor Laboratory logo. The bioRxiv logo is prominently displayed with the tagline 'THE PREPRINT SERVER FOR BIOLOGY'. Below this, the text 'New Results' is on the left and a 'Follow this preprint' button is on the right. The title of the preprint is 'Transformers do not outperform Cellpose'. The authors are listed as 'Carsen Stringer' and 'Marius Pachitariu'. A DOI link is provided: 'https://doi.org/10.1101/2024.04.06.587952'. A disclaimer states: 'This article is a preprint and has not been certified by peer review [what does this mean?].'. Below the text are social media sharing icons for various platforms. At the bottom, there are tabs for 'Abstract', 'Info/History', and 'Metrics', and a 'Preview PDF' button.

Abstract

In a recent publication, Ma et al (2024) claim that a transformer-based cellular segmentation method called Mediar - which won a Neurips challenge - outperforms Cellpose (0.897 vs 0.543 median F1 score). Here we show that this result was obtained by artificially impairing Cellpose in multiple ways. When we removed these impairments, Cellpose outperformed Mediar (0.861 vs 0.826 median F1 score on the updated test set). To further investigate the performance of transformers for cellular segmentation, we replaced the Cellpose backbone with a transformer. The transformer-Cellpose model also did not outperform the standard Cellpose (0.848 median F1 test score). Our results suggest that transformers do not advance the state-of-the-art in cellular segmentation.

Competing Interest Statement

The authors have declared no competing interest.

Copyright The copyright holder for this preprint is the author/funder, who has granted bioRxiv a license to display the preprint in perpetuity. It is made available under a CC-BY-NC 4.0 International license.

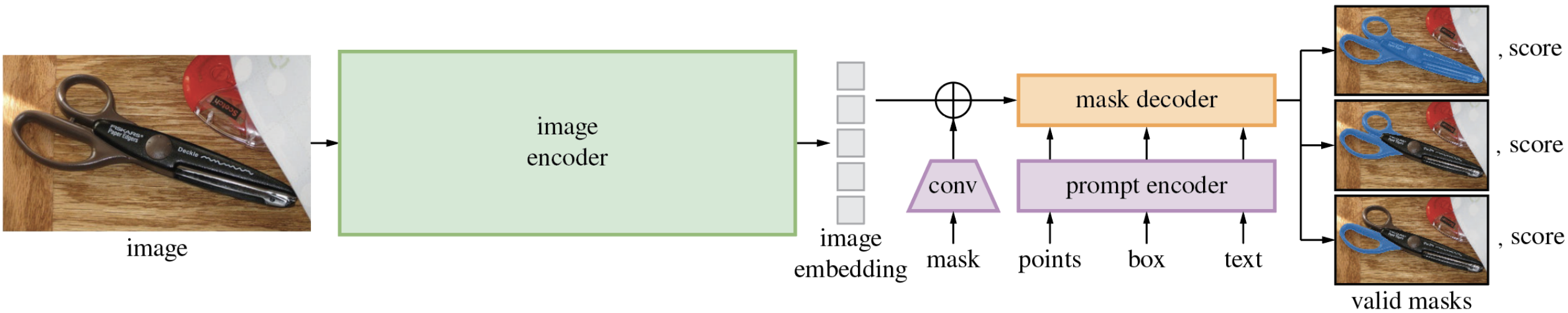
Segment Anything Model

New approach to DL-based image segmentation involving prompts



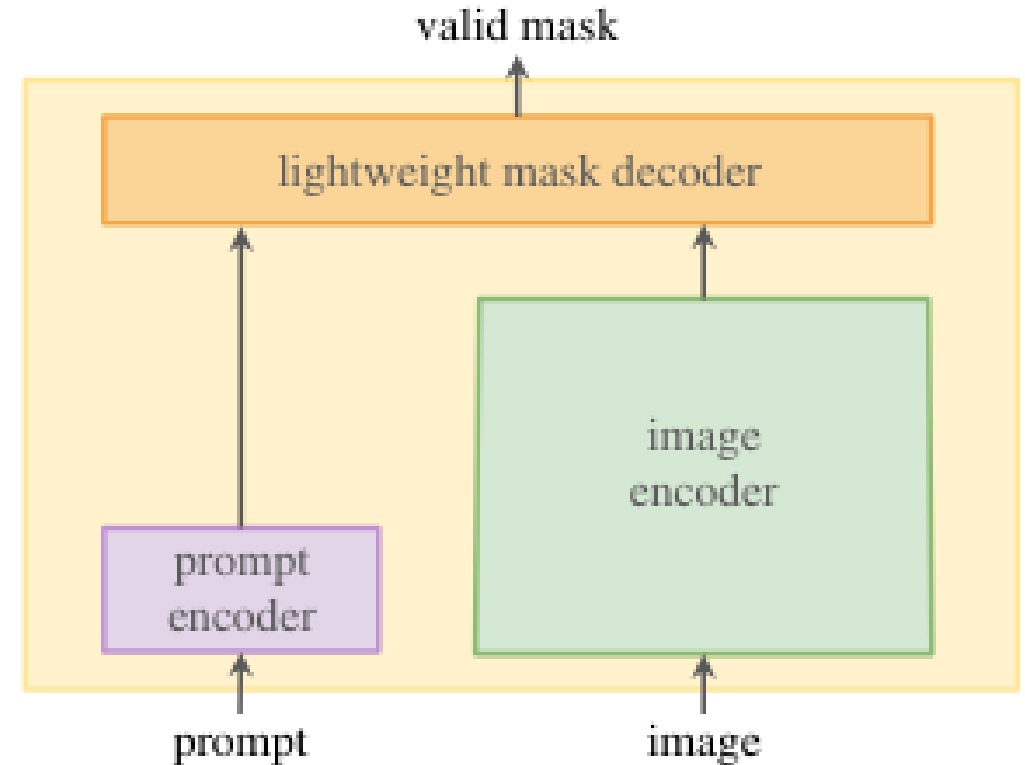
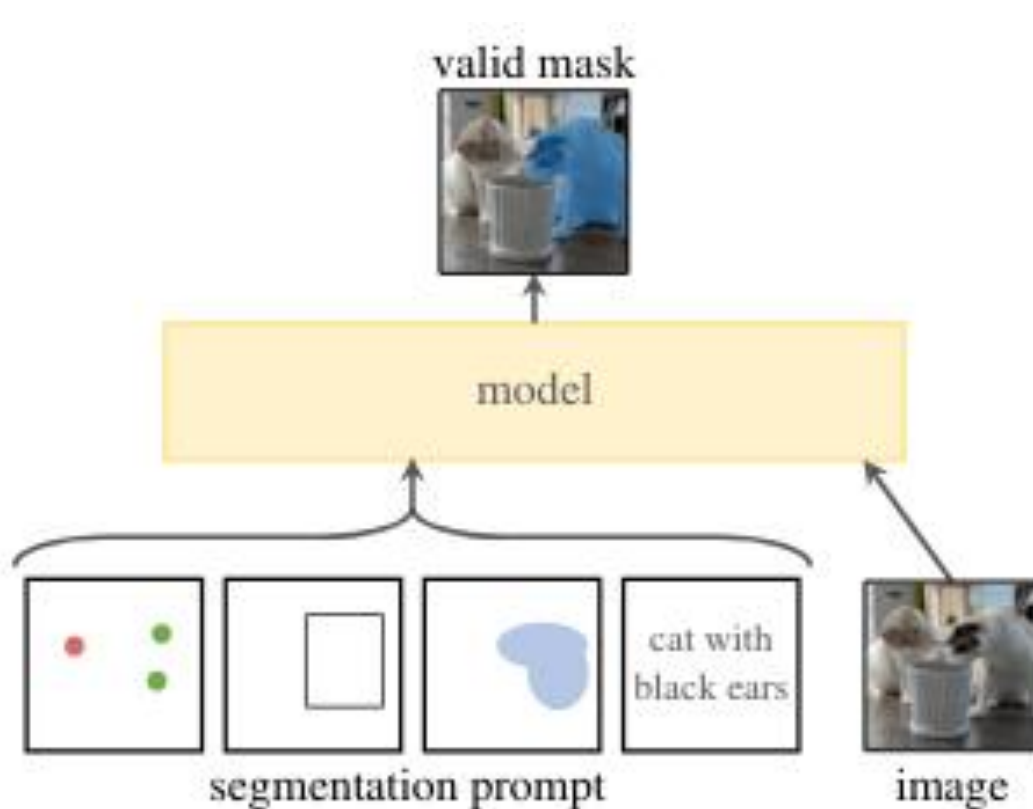
Segment Anything Model

New approach to DL-based image segmentation involving prompts



Segment Anything Model

New approach to DL-based image segmentation involving prompts



Segment Anything Model

Trained on mostly natural images

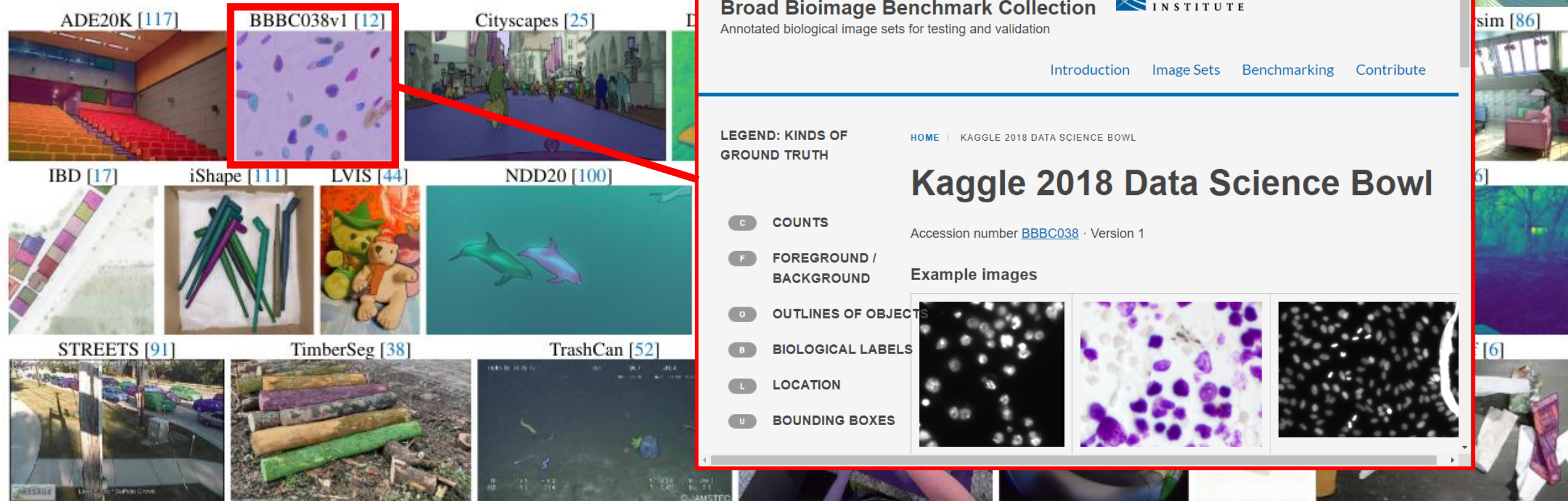


Figure 8: Samples from the 23 diverse segmentation datasets used to evaluate SAM's zero-shot transfer capabilities.

Segment Anything Model

Downsides:

- Original code did not contain the procedure for iterative training
- Instance segmentation not ideal (watershed-based implementation added in micro-sam)
- Fine-tuning for microscopy data seems necessary

Segment Anything for Microscopy

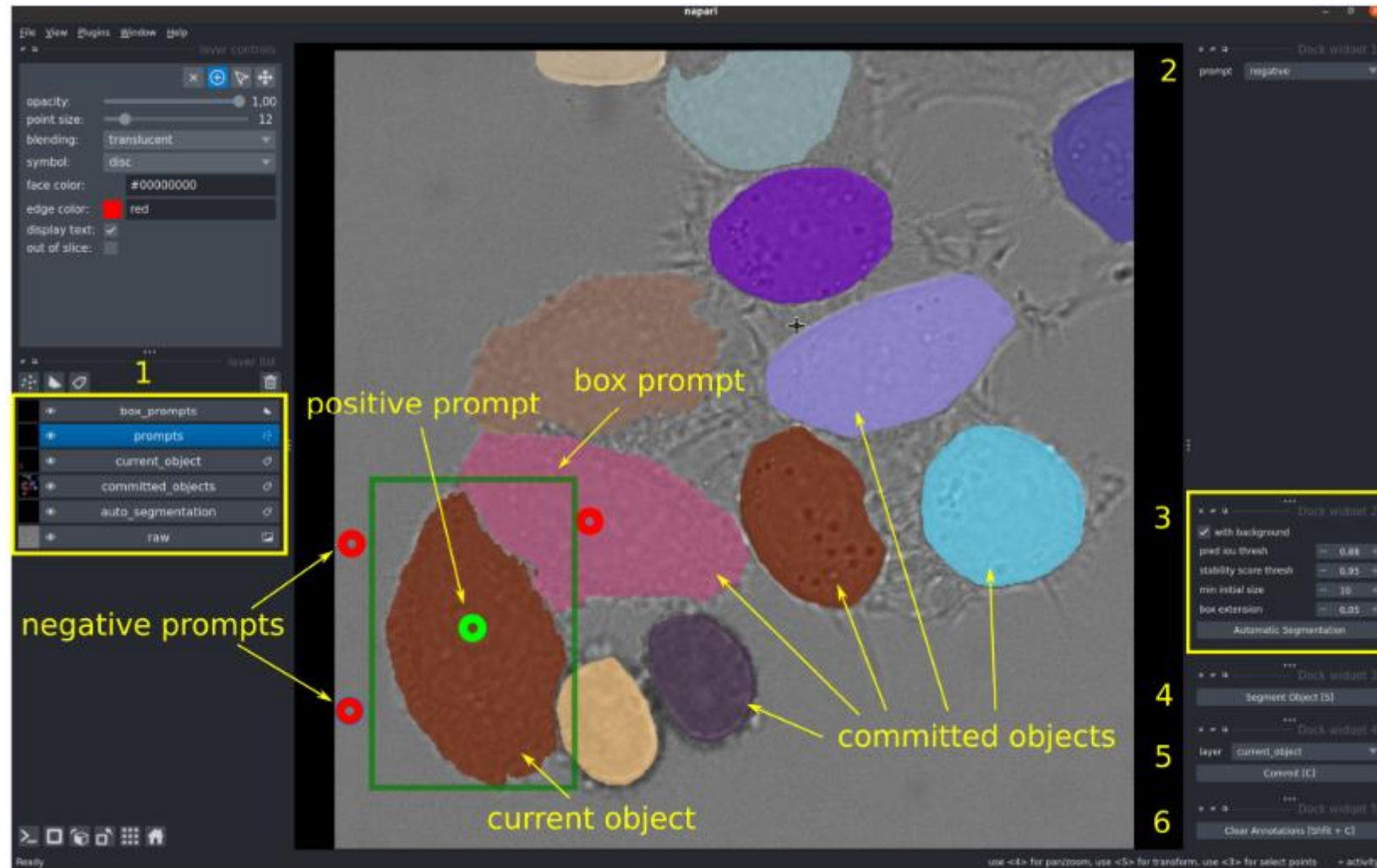
Popping up napari plugins, some within 24h after SAM was published

The image displays four browser windows showing GitHub repositories for napari plugins related to Segment Anything (SAM) for microscopy. Each window shows the repository's README, license, and a video demonstration of the plugin's capabilities.

- Window 1: hiroalchem/napari-SAM4IS**
 - Repository: github.com/hiroalchem/napari-SAM4IS
 - License: Apache-2.0
 - Python versions: 3.8 | 3.9 | 3.10
 - Tests: failing
 - Description: napari plugin for instance and semantic segmentation using Segment Anything Model (SAM).
 - Installation: `pip install "napari[all]"`
- Window 2: royerlab/napari-segment-anything**
 - Repository: github.com/royerlab/napari-segment-anything
 - License: Apache-2.0
 - Python versions: 3.8 | 3.9 | 3.10
 - Tests: failing
 - Description: Napari plugin of Segment Anything Model (SAM).
 - Download the network weights [here](#).
- Window 3: MIC-DKFZ/napari-sam**
 - Repository: github.com/MIC-DKFZ/napari-sam
 - License: Apache-2.0
 - Python versions: 3.8 | 3.9 | 3.10
 - Tests: failing
 - Description: Segment anything with our Napari integration of Meta AI's new Segment Anything Model (SAM).
 - Features: SAM is the new segmentation system from Meta AI capable of one-click segmentation of any object, and now, our plugin neatly integrates this into Napari. We have already extended SAM's click-based foreground separation to full click-based semantic segmentation and instance segmentation!
 - Supports both 2D and 3D images!
- Window 4: computational-cell-analytics/micro-sam**
 - Repository: github.com/computational-cell-analytics/micro-sam
 - License: MIT
 - Python versions: 3.8 | 3.9 | 3.10
 - Tests: failing
 - Description: Segment Anything for Microscopy. Tools for segmentation and tracking in microscopy build on top of Segment Anything and track objects in microscopy images interactively with a few clicks!
 - We implement napari applications for:
 - interactive 2d segmentation (Left: interactive cell segmentation)
 - interactive 3d segmentation (Middle: interactive mitochondria segmentation)
 - interactive tracking of 2d image data (Right: interactive cell tracking)

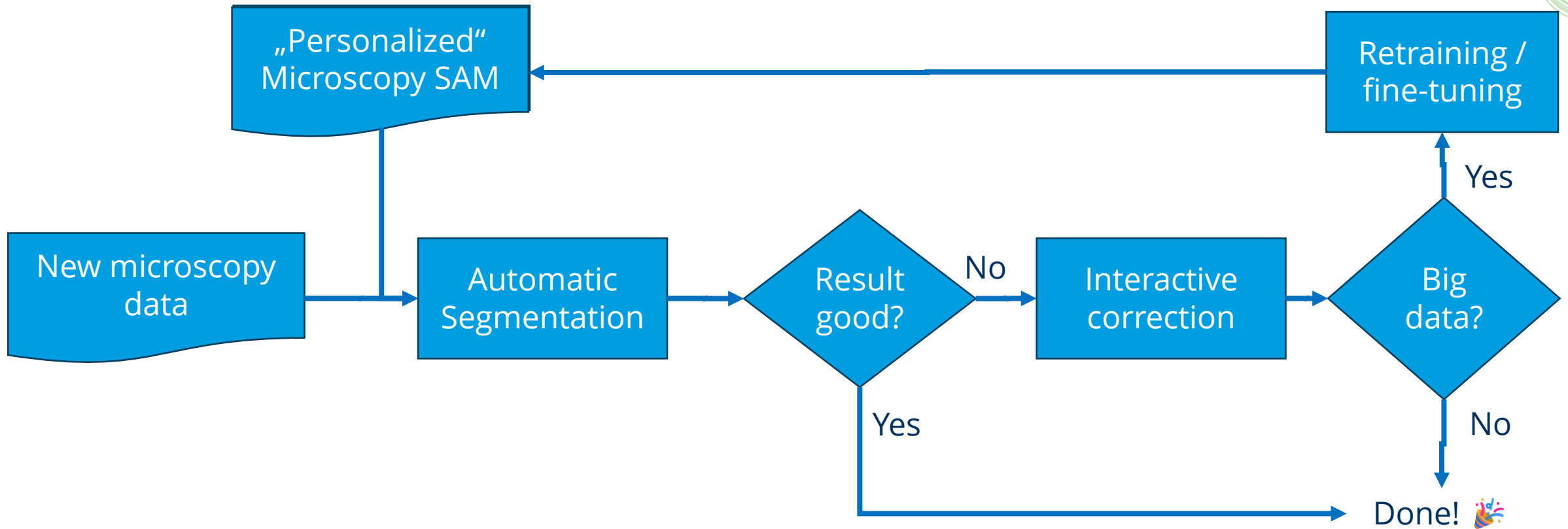
Segment Anything for Microscopy

Micro-sam

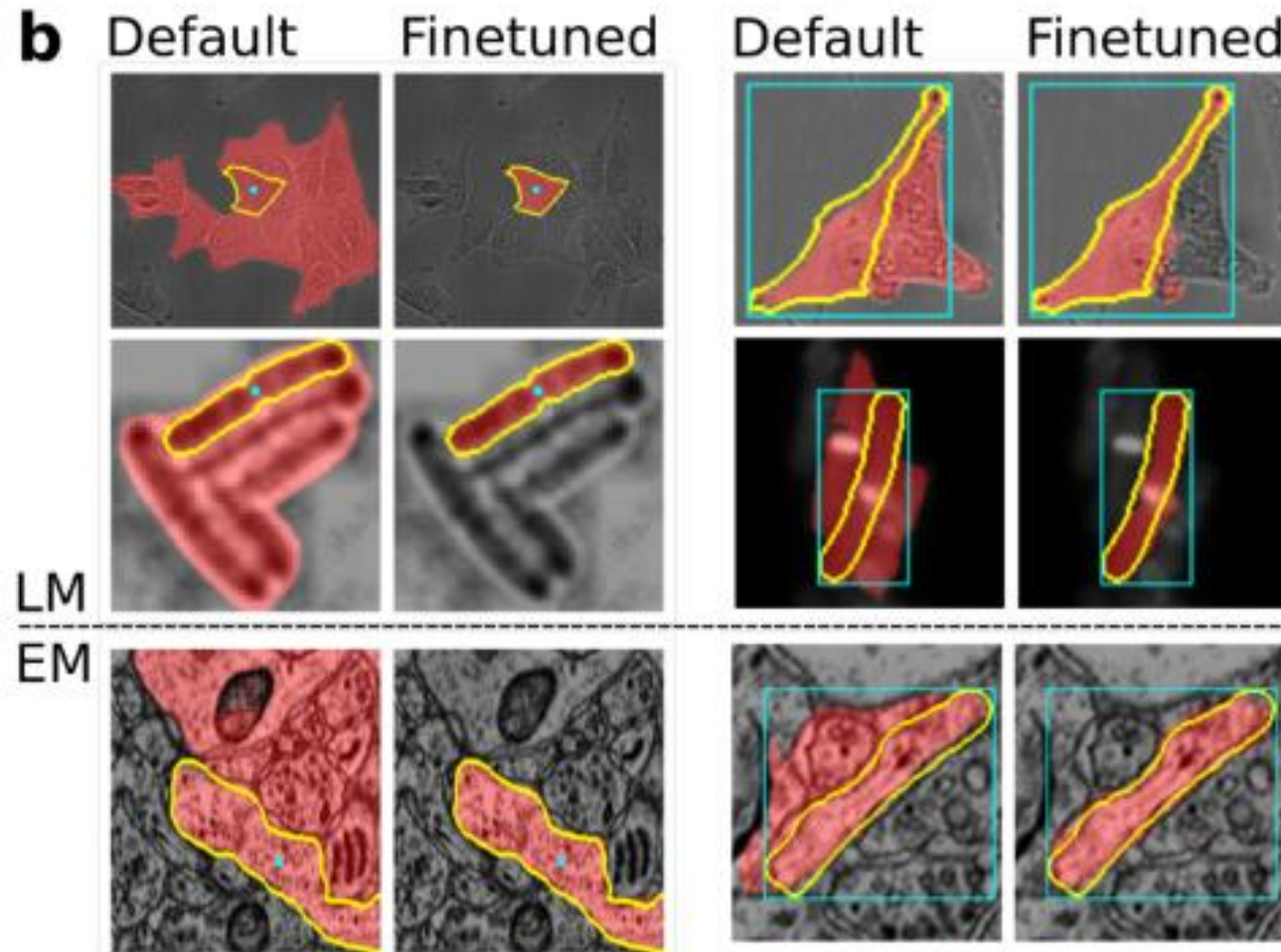


Segment Anything for Microscopy

Real-world scenario: human-in-the-loop



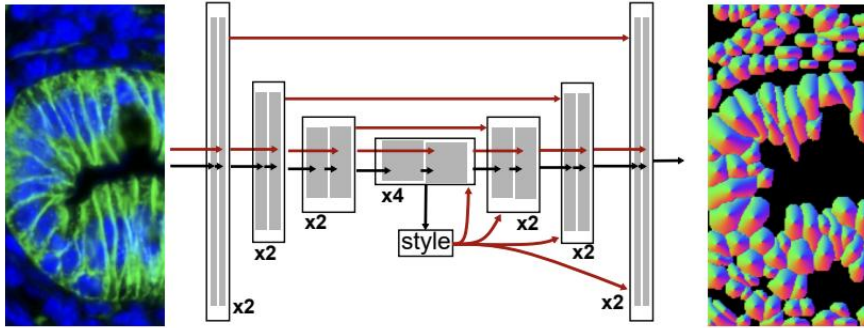
Segment Anything for Microscopy



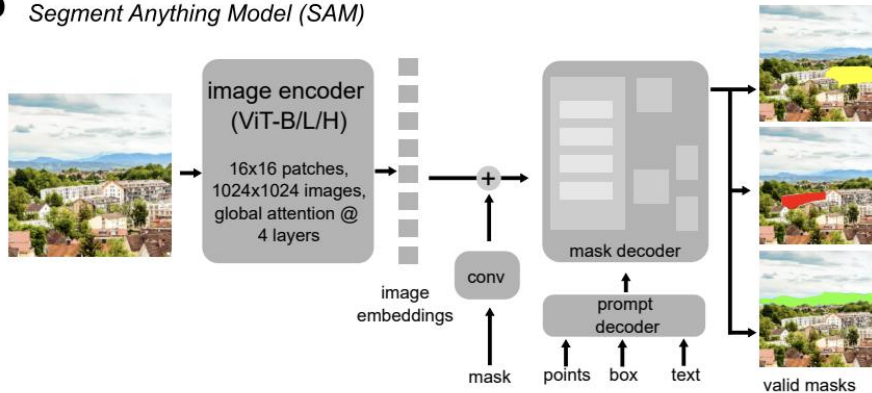
CellPose-SAM

Recent combination of CellPose + Segment Anything Model (May 1st 2025)

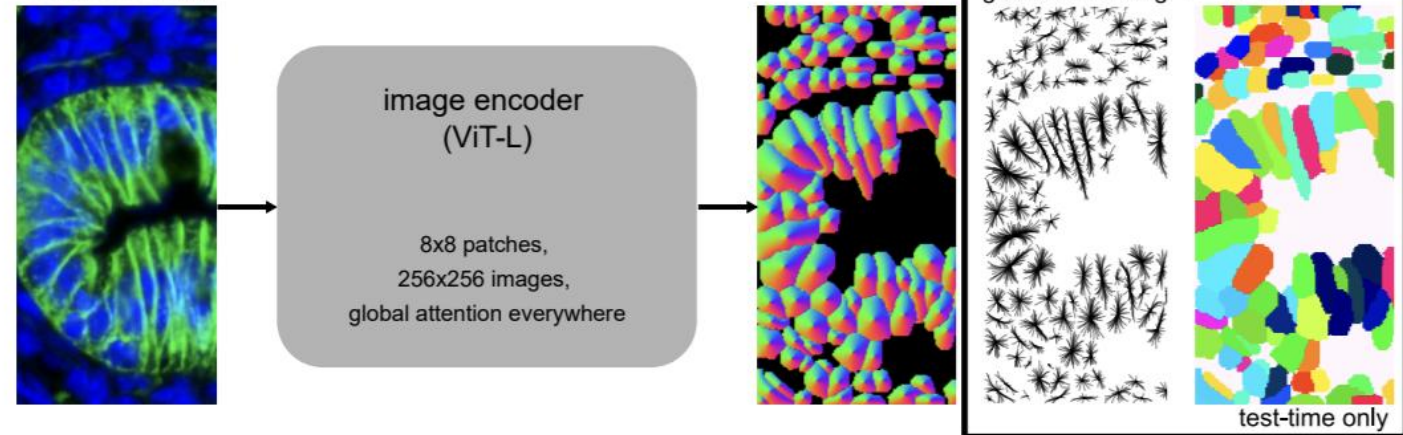
a Cellpose



b Segment Anything Model (SAM)



c Cellpose-SAM



CellPose-SAM: Model fine-tuning

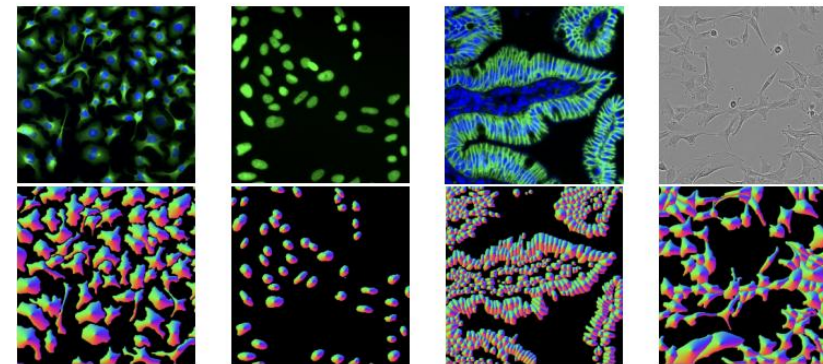
Model fine-tuning to make it a microscopy-specific image processing model

Training dataset for SAM (SA-1B)

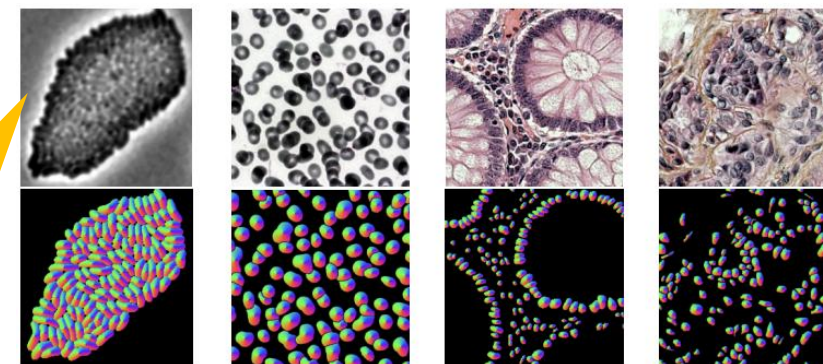


300.000 images, 10.2 million manual ROIs

Finetuning Cellpose-SAM on cellular datasets



Based on publicly available benchmark data (Grand Challenges etc.)



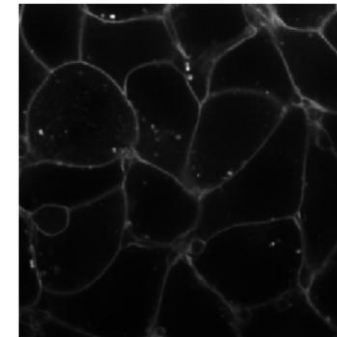
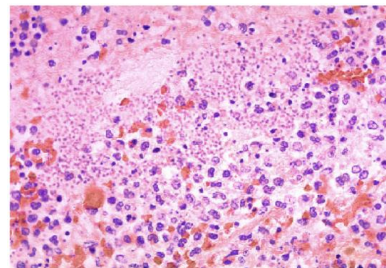
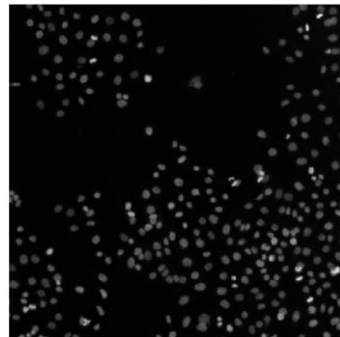
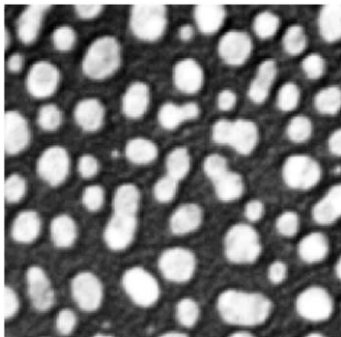
22, 826 images, 3.34 million manual ROIs



Quiz

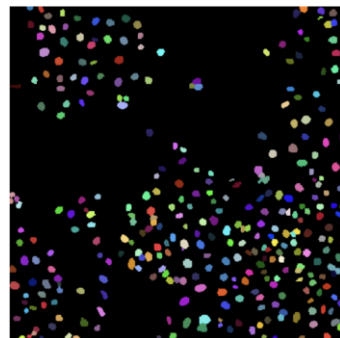
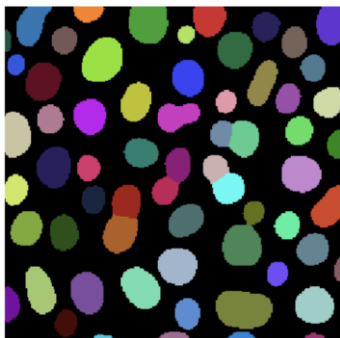
If I train a model on publicly available benchmark data, what are advantages and disadvantages of the resulting model?

CellPose SAM in Python

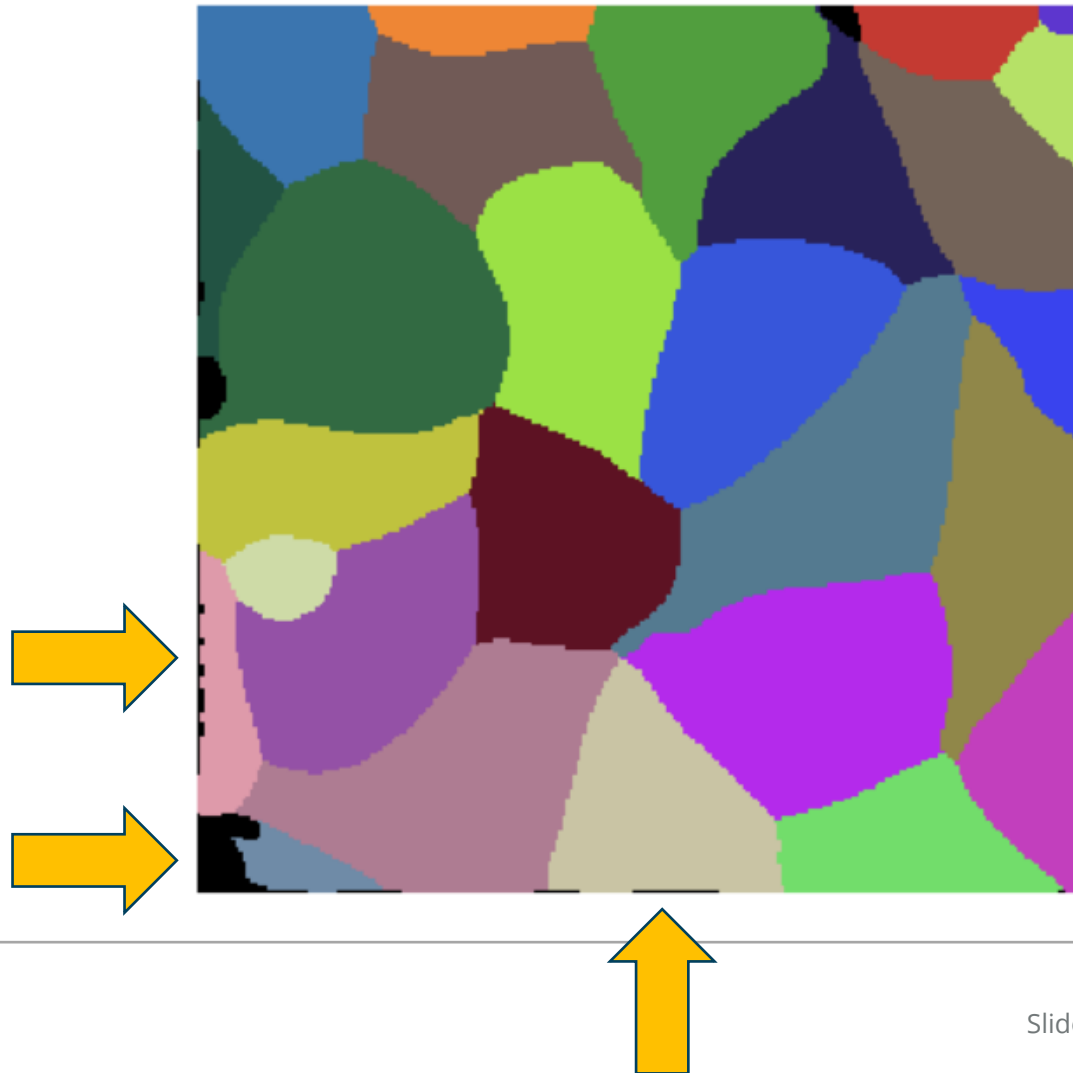


```
masks, flows, styles = model.eval(image,  
    batch_size=32,  
    flow_threshold=0.4,  
    cellprob_threshold=0.0,  
    normalize={"tile_norm_blocksize": 0})  
  
stackview.insight(masks.astype(np.uint32))
```

“Foundation
model”



Quiz: What could we do to deal with such artifacts at the image border?



Summary: Deep Learning for Bio-image Analysis

- [Convolutional] Neural Networks is a decade old technology that enabled breakthroughs recently.
 - Image Denoising
 - Image Segmentation
- Common scheme: Smart algorithms for processing input/output of neural networks + standard NN architectures
 - ~~Image in, instance segmentation out~~
- Training these models is
 - computationally expensive,
 - needs large amounts of training data images / full annotations (~~single images, sparse annotations~~),
 - requires a certain level of [python] expertise
- If Voronoi-Otsu-Labeling does the job, don't dive into deep learning!

Exercises

Robert Haase

Funded by



Bundesministerium
für Bildung
und Forschung

SACHSEN



Diese Maßnahme wird gefördert durch die Bundesregierung
aufgrund eines Beschlusses des Deutschen Bundestages.
Diese Maßnahme wird mitfinanziert durch Steuermittel auf
der Grundlage des von den Abgeordneten des Sächsischen
Landtags beschlossenen Haushaltes.

Exercises

Make noise2void, stardist, cellpose, micro-sam and cellpose-sam work.

Hint: This may screw up your conda environment.

In case of weird errors:

1. Don't panic
2. Recreate you environment
3. Install the thing you want to use, preferably using conda.

Creating conda environments is not a big deal, just like grabbing a spoon before eating soup.

```
Command Prompt - pip install n2v tensorflow==2.5.1
matplotlib, xarray, google-auth-oauthlib, csbdeep, bioimageio.s
Attempting uninstall: wrapt
Found existing installation: wrapt 1.16.0
Uninstalling wrapt-1.16.0:
  Successfully uninstalled wrapt-1.16.0
Attempting uninstall: typing-extensions
Found existing installation: typing_extensions 4.11.0
Uninstalling typing_extensions-4.11.0:
  Successfully uninstalled typing_extensions-4.11.0
Attempting uninstall: six
Found existing installation: six 1.16.0
Uninstalling six-1.16.0:
  Successfully uninstalled six-1.16.0
Attempting un
Found existi
Uninstalling
  Successful
Attempting un
Found existi
Uninstalling
  Successful
Attempting un
Found existi
Uninstalling
  Successful
Attempting un
Found existi
Uninstalling
  Successful
```

```
Command Prompt
Uninstalling matplotlib-3.8.4:
  Successfully uninstalled matplotlib-3.8.4
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behavior
is the source of the following dependency conflict:
google-cloud-speech 1.30.1 requires shapely<1.0.0dev, which is not installed.
grpc-google-iam-v1 0.13.0 requires grpcio<2.0.0dev,>=1.44.0, but you have grpcio 1.34.1 which is incompatible.
grpcio-status 1.60.0 requires grpcio<=1.60.0, but you have grpcio 1.34.1 which is incompatible.
grpcio-status 1.60.0 requires protobuf<=4.21.6, but you have protobuf 3.20.3 which is incompatible.
aiosqlite 0.20.0 requires typing_extensions<=4.0, but you have typing_extensions 3.7.4.3 which is incompatible.
apoc 0.12.0 requires numpy>=1.21, but you have numpy 1.19.5 which is incompatible.
devbio-napari 0.10.1 requires numpy>=1.21.4, but you have numpy 1.19.5 which is incompatible.
hdbscan 0.8.33 requires numpy>=1.20, but you have numpy 1.19.5 which is incompatible.
magicgui 0.8.2 requires typing_extensions>=4.1.0, but you have typing_extensions 3.7.4.3 which is incompatible.
napari 0.4.18 requires numpy>=1.21, but you have numpy 1.19.5 which is incompatible.
napari 0.4.18 requires typing_extensions>=4.2.0, but you have typing_extensions 3.7.4.3 which is incompatible.
napari-animation 0.0.8 requires napari>=0.4.19rc5, but you have napari 0.4.18 which is incompatible.
napari-clusters-plotter 0.7.3 requires numpy<=1.23.5,>=1.21, but you have numpy 1.19.5 which is incompatible.
```

```
[1]: import n2v
from skimage.io import imread
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[1], line 2
      1 import n2v
----> 2 from skimage.io import imread
```

```
File ~\mambaforge\envs\tea2\lib\site-packages\skimage\__init__.py:122
    118 # We are not importing the rest of the scikit during the build
    119 # process, as it may not be compiled yet
    120 else:
    121     try:
--> 122         from ._shared import geometry
    123         del geometry
    124     except ImportError as e:
```

```
File geometry.pyx:1, in init skimage._shared.geometry()
```

```
ValueError: numpy.ndarray size changed, may indicate binary incompatibility. Expected 96 from C header,
got 80 from PyObject
```